PSC INF : RAPPORT FINAL

Machine Learning Quantique dans la Détection de Fraude Bancaire

Avril 2026

Hannah Faucheu, Guillaume Henrotte, Achim Kobena Dongo, Tancrède Tonini, Kristen Boitier, Stanislas Mischler

TABLE DES MATIÈRES

1 Introduction **3**
1.1 Motivation générale . . . 3
1.2 État de l'art . . . 3
1.2.1 Approches classiques de référence . . . 4
1.2.2 Clustering quantique . . . 4
1.2.3 Classifieurs QML supervisés . . . 4

2 Présentation du projet **4**
2.1 Méthode générale . . . 4
2.2 Données utilisées . . . 5

3 Données de la BNP **6**

4 Techniques de preprocessing **6**

5 Algorithmes non supervisés **7**
5.1 Clustering classique . . . 7
5.2 Clustering quantique variationnel . . . 7
5.3 Implémentation . . . 9
5.4 Rappels théoriques : SVM classique et astuce du noyau . . . 16
5.5 Quantum One-Class Support Vector Machine (Q-OCSVM) . . . 17

6 Réseaux à couches **20**

7 Réseaux à de neurones **20**
7.1 Réseaux de neurones adversariaux . . . 20
7.1.1 Théorie . . . 21
7.1.2 QGANs pour la détection de Fraude . . . 21

8 Réseux à couche **22**
8.1 Motivation et positionnement dans la pipeline . . . 22
8.2 Principe de la détection en cascade . . . 23
8.3 Description des trois couches classiques . . . 24
8.4 Résultats de la cascade classique . . . 24
8.5 Extension quantique de la cascade . . . 25
8.5.1 Pourquoi insérer une couche quantique dans la cascade ? . . . 25
8.5.2 Quelles couches remplacer ? . . . 25
8.5.3 Analyse de faisabilité et limites . . . 26
8.5.4 Architecture de la cascade hybride proposée . . . 27
8.6 Positionnement dans l'architecture globale . . . 28

9 Assemblage d'une pipeline **28**
9.1 Architecture modulaire . . . 28
9.2 Résultats . . . 28

10 Tests et Comparaisons **28**
10.1 Comparaison de 4 algorithmes sur un dataset tiers . . . 29
10.2 Application au jeu de données de la BNP . . . 29

11 Difficultés et critique **30**
11.1 Limites matérielles 30
11.2 Absence de labels dans le jeu de données de la BNP 30

12 Conclusion et idées futures **31**
12.1 Bilan 31
12.2 Idées futures 31

1 INTRODUCTION

La détection de fraude bancaire est aujourd'hui l'un des enjeux opérationnels majeurs des grandes institutions financières. Le volume toujours croissant des transactions, la diversification des canaux de paiement et l'adaptation permanente des comportements frauduleux rendent la tâche particulièrement difficile pour les approches classiques, qui doivent composer avec des jeux de données massifs, déséquilibrés et en constante évolution. Dans ce contexte, l'informatique quantique, et plus particulièrement le *Quantum Machine Learning* (QML), apparaît comme une voie prometteuse pour dépasser certaines limites théoriques des algorithmes classiques.

Ce rapport final s'inscrit dans la continuité de nos travaux menés en collaboration avec BNP Paribas dans le cadre du PSC INF. Il présente une synthèse de l'ensemble de nos travaux : les algorithmes que nous avons étudiés, les implémentations que nous avons mises en place, ainsi que leur confrontation aux données réelles fournies par la banque.

1.1 Motivation générale

La détection de fraude constitue un enjeu à la fois économique, réglementaire et technique pour les institutions bancaires. Chaque transaction frauduleuse non détectée se traduit par une perte financière directe, une atteinte à la confiance des clients et une exposition accrue aux contraintes de conformité. À l'inverse, un taux trop élevé de faux positifs dégrade l'expérience utilisateur et mobilise inutilement les équipes d'analyse. Les méthodes classiques de *machine learning*, bien que matures, atteignent progressivement leurs limites face à la croissance du volume et de la complexité des données financières, ainsi qu'à la nature intrinsèquement mouvante de la fraude.

Les algorithmes quantiques promettent, du moins sur le plan théorique, des gains significatifs en complexité et en expressivité par rapport à leurs équivalents classiques, notamment pour des tâches de clustering, de détection d'anomalies ou de recherche dans des espaces de grande dimension. Cette promesse reste toutefois largement conditionnée par la maturité du matériel actuel — processeurs NISQ bruités, nombre limité de qubits, absence de QRAM opérationnelle — et par la capacité à concevoir des algorithmes véritablement adaptés aux contraintes du domaine bancaire.

Nos objectifs sont d'explorer les différentes méthodes de *machine learning* quantique pertinentes pour ce problème, de les comparer entre elles et à leurs contreparties classiques, et d'aboutir à une pipeline complète de détection de fraude. Le but final est de fournir à BNP Paribas une méthode hybride classique/quantique opérationnelle pour la détection de fraude bancaire.

1.2 État de l'art

La détection de fraude est un problème classique de classification supervisée fortement déséquilibrée, pour lequel la littérature classique est abondante (régressions logistiques, gradient boosting, forêts aléatoires, autoencodeurs, réseaux de neurones profonds). Notre objectif n'est pas de rivaliser frontalement avec ces méthodes,

mais d'examiner dans quelle mesure des algorithmes quantiques, ou hybrides classique/quantique, peuvent apporter un gain *théorique* (complexité, expressivité) ou *pratique* (qualité de détection) sur ce type de données. Nous distinguons trois familles de travaux qui structurent notre état de l'art : les approches classiques de référence, le clustering quantique, et les classifieurs QML supervisés.

1.2.1 • Approches classiques de référence

Les méthodes classiques déployées en production reposent essentiellement sur des modèles supervisés entraînés sur des historiques étiquetés. Le gradient boosting (XGBoost, LightGBM) et les forêts aléatoires constituent aujourd'hui les *baselines* usuelles, car ils gèrent bien les variables hétérogènes et offrent une bonne robustesse au bruit. Pour les jeux de données fortement déséquilibrés comme ceux de la fraude (où la classe positive représente typiquement moins de 0,2% des transactions), l'emploi de techniques de rééchantillonnage telles que SMOTE [4] et de prétraitements par ACP est devenu standard. En parallèle, les méthodes non supervisées — Isolation Forest, Local Outlier Factor, auto-encodeurs — sont utilisées soit pour pallier l'absence d'étiquettes fiables, soit pour construire des scores d'anomalie utilisés en amont d'un classifieur supervisé. Ces approches constituent le point de comparaison incontournable pour tout algorithme quantique prétendant améliorer la détection de fraude.

1.2.2 • Clustering quantique

La question du clustering quantique a commencé à être évoquée il y a quelques années, avec la popularisation du terme de *Quantum Machine Learning* (QML) [8]. Les limites actuelles des ordinateurs disponibles au grand public, en matière de taille comme de fiabilité (on parle de *Noisy Intermediate-Scale Quantum* (NISQ) devices), empêchent toute forme d'implémentation fonctionnelle intégralement quantique.

Nous pouvons noter qu'il existe deux articles auxquels nous ferons référence pour le clustering avec une approche purement quantique : Kerenidis et al. [11] pour un algorithme q-means, et Kerenidis et Landman [12] pour un algorithme de clustering spectral quantique. Ces articles reposent cependant sur l'utilisation d'une QRAM, dont la réalisation physique efficace reste un problème ouvert et fait débat. Beaucoup d'articles se rabattent donc sur une approche hybride quantique/classique, exploitant l'architecture quantique pour certains calculs uniquement : Poggiali et al. [14] proposent par exemple une telle implémentation de k-means, tandis que Bermejo et Orús [13] présentent une approche variationnelle du clustering quantique.

1.2.3 • Classifieurs QML supervisés

Les classifieurs QML supervisés reposent majoritairement sur les *Variational Quantum Classifiers* et les *Quantum Neural Networks* : un *feature map* encode la donnée $\vec{x}$ dans un état quantique, un ansatz paramétré $U(\theta)$ est ensuite appliqué, et la mesure d'une observable fournit un score optimisé classiquement (COBYLA, SPSA, Adam). Le *ZZ feature map* introduit par Havlíček et al. [10] est devenu le cadre de référence pour exploiter les corrélations entre variables dans l'espace de Hilbert.

2
PRÉSENTATION DU PROJET

2.1 Méthode générale

Notre démarche a débuté par une phase d'*exploration horizontale* : nous avons balayé plusieurs familles d'algorithmes candidats, y compris des pistes plus prospectives comme l'étude des transactions par transformée de Fourier quantique. Cette voie s'est révélée peu concluante — résultats difficilement interprétables, littérature

scientifique insuffisante sur le sujet appliqué à la fraude — et nous avons choisi de la mettre de côté pour concentrer nos efforts sur des briques mieux maîtrisées théoriquement.

Cette exploration nous a menés à approfondir deux axes quantiques identifiés à partir d'une revue de la littérature : le *clustering quantique variationnel* (VQC) et le *Quantum One-Class SVM* (Q-OCSVM). À ces deux briques héritées de la littérature QML, nous ajoutons deux contributions propres à notre groupe : d'une part une étude approfondie du *préprocessing* (SMOTE, Tomek Links, ACP, RFE) et de son impact sur les performances, d'autre part la conception d'une architecture en *cascade hybride* (réseaux à couches) qui enchaîne plusieurs classifieurs de complexité croissante, dans la continuité directe de notre objectif final : aboutir à une pipeline générale de détection de fraude pour BNP Paribas.

Nous reprenons l'approche *plug & play* initiée par le groupe 2024-25 : chaque brique (préprocessing, clustering, détection d'anomalies, classifieur) est implémentée comme un composant interchangeable respectant une interface commune, ce qui permet de composer et de comparer des configurations entièrement classiques, entièrement quantiques, ou hybrides sans réécrire le code global.

Déroulement du travail. Notre projet s'est structuré en trois phases. La première, *individuelle*, a été consacrée à la familiarisation avec l'informatique quantique : visionnage des vidéos pédagogiques d'IBM, lecture de *Quantum Computation and Quantum Information* de Nielsen et Chuang, recherche et synthèse d'articles pertinents pour notre problème. Dans un deuxième temps, nous nous sommes répartis en *axes de travail* parallèles, chaque sous-groupe prenant en charge l'étude théorique et l'implémentation d'une brique. La troisième phase a consisté en une *mise en commun* : agencement des briques les unes avec les autres, benchmarks comparatifs et évaluation de la pipeline vis-à-vis de notre objectif final.

Traitement séparé des données BNP. Les données fournies par BNP Paribas nécessitent un *préprocessing* conséquent — variables catégorielles à forte cardinalité, absence de labels, volumétrie importante — qui en a fait un chantier à part entière. Nous les avons donc traitées de manière séparée pendant la majeure partie du projet, et validé nos algorithmes sur des jeux alternatifs publiquement disponibles (Kaggle, *breast cancer*, etc., détaillés plus tard), qui offrent une vérité terrain exploitable pour comparer rigoureusement nos briques.

Environnement technique. Toutes nos expériences ont été menées sur simulateur : Qiskit (`AerSimulator`) et PennyLane, pilotés depuis Python. Nous n'avons pas fait appel au hardware IBM, ce qui nous autorise des tailles d'instance et des budgets de *shots* supérieurs à ceux accessibles sur processeur réel, au prix de l'absence de bruit matériel — limite que nous discutons dans le chapitre consacré aux difficultés et critiques.

2.2 Données utilisées

Nous avons travaillé sur trois jeux de données complémentaires.

- **Données BNP Paribas.** Jeu cible de notre étude, caractérisé par des variables catégorielles à forte cardinalité (identifiants de traders, localisations) qui ne se prêtent pas directement aux algorithmes géométriques. Nous lui consacrons une section dédiée (3) pour leur prétraitement.
- ***Credit Card Fraud Detection*** [19]. Jeu public Kaggle de 284 807 transactions européennes (septembre 2013), dont 492 fraudes (0,17%, fortement déséquilibré). Features anonymisées par ACP ($V_1, \ldots, V_{28}$) plus *Time* et *Amount*, avec labels binaires : une vérité terrain nette nous permettant d'implémenter certains de nos algorithmes.
- ***Breast Cancer Wisconsin*** [20]. Jeu public Kaggle de 569 diagnostics de tumeurs (357 bénignes, 212 malignes) caractérisées par 30 variables numériques. De petite taille et de classes relativement équilibrées, il nous sert de terrain de benchmark rapide pour comparer nos briques classiques et quantiques.

3
DONNÉES DE LA BNP

4
TECHNIQUES DE PREPROCESSING

Le but du machine learning est de faire de l'apprentissage à partir de notre jeu de données. Cependant, une étape clé en amont de l'apprentissage est de modifier ce jeu de données pour le rendre plus exploitable, plus performant, plus apte à révéler de l'information. Ainsi nous présentons plusieurs méthodes dites de Preprocessing pour optimiser notre jeu de données : SMOTE, Tomek Links, PCA et RFE. Voici en quoi ils consistent et quel est leur intérêt :

— SMOTE (Synthetic Minority Over-sampling TEchnique) : concerne l'apprentissage supervisé, c'est-à-dire qu'il nécessite des données labellisées (fraude ou non). L'idée est que, dans un cas qui arrive souvent où dans nos labels il y a peu de données frauduleuses, alors l'apprentissage est peu efficace car il compare souvent des données non frauduleuses entre elles et les cas de fraude sont donc rares, donc plus enrichissants. Ainsi le but est de rééquilibrer les proportions artificiellement. Pour la détection de fraude, on a souvent moins de 0.1 % des données qui sont frauduleuses. L'idée est de plutôt s'approcher d'un ratio 99 % - 1% ou 95 % - 5%. Pour cela, on prend une paire de données frauduleuses et on ajoute à notre base de données une donnée frauduleuse qui se trouve à leur milieu dans le plan. On peut également choisir un barycentre différent ou le prendre de manière aléatoire. L'idée étant que notre nouveau point est proche de données frauduleuses, il est donc très suspect d'être frauduleux.

— Tomek Links : concerne également l'apprentissage supervisé mais agit à l'inverse de SMOTE pour supprimer quelques points du plan pour optimiser l'apprentissage. Dans un cas comme le nôtre, où il y a une dissymétrie brutale entre le nombre de labels fraude et non fraude, on cherche à préserver au mieux nos données les plus intéressantes qui sont celles ayant le label fraude. Ainsi on élimine du plan les points non fraude qui sont trop proches des points fraude. On dit qu'une paire de points du plan forme un Tomek Link si chacun est le plus proche voisin de l'autre. Si on a une paire fraude-non fraude qui forme un Tomek Link, on supprime le point non fraude. La surreprésentation des points non fraude fait que la perte n'est pas significative tandis qu'on protège l'information contenue dans les labels fraude. On peut ensuite tout à fait itérer ce procédé.

— PCA (Principal Component Analysis) : cet algorithme réduit le nombre de paramètres, c'est-à-dire la dimension du plan, et bascule dans un nouveau système de coordonnées pour optimiser l'accessibilité à l'information de la base de données. La trentaine de paramètres initiaux est superflue. On ne veut garder que les éléments les plus intéressants pour notre apprentissage et on dispose d'un critère simple de sélection. On calcule la matrice de covariance de nos paramètres. Puis on sélectionne le sous-espace propre associé aux plus grandes valeurs propres (plus grandes variances, donc avec le moins de redondance), ce qui est immédiat à faire après diagonalisation. Les vecteurs propres associés que nous prenons pour nouveaux paramètres sont orthonormés, ce qui garantit l'indépendance et la singularité des informations qu'on peut y exploiter.

— RFE (Recursive Feature Elimination) : cet algorithme tout comme le précédent vise à réduire le nombre de paramètres. On applique un algorithme de machine learning sur notre jeu de données pour faire notre apprentissage (par exemple l'arbre de décision) puis on sélectionne le paramètre ayant un pouvoir de décision le plus faible, c'est-à-dire celui qui est le moins retenu quand on décide de la nature (fraude ou non) d'un nouveau point. Ce paramètre est ensuite éliminé et on recommence notre apprentissage de zéro. L'idée est d'itérer ce procédé pour finir avec un nombre restreint de paramètres (environ une dizaine).

Ordre de la pipeline preprocessing : l'ordre dans lequel on applique les algorithmes a son importance, ils ont été présentés dans l'ordre de la pipeline, voici pourquoi :

— SMOTE puis Tomek Links. Le SMOTE crée des données synthétiques entre les points existants. Le problème, c'est qu'il crée souvent des points bruités qui empiètent sur la classe majoritaire. En passant les Tomek Links après, on supprime les points qui sont trop proches les uns des autres entre les deux classes. Cela nettoie la frontière de décision que le SMOTE a rendue floue.
— PCA ensuite. La PCA cherche à maximiser la variance. Si on fait la PCA avant de rééquilibrer les classes, elle sera totalement biaisée par la classe majoritaire. En faisant la PCA après le rééquilibrage et le nettoyage, on s'assure que les composantes principales représentent bien la structure des deux classes de manière équitable.
— RFE après PCA. RFE est très coûteux car il nécessite d'entraîner l'algorithme à chaque fois. Ainsi on l'utilise pour affiner. Le seul bémol d'utiliser RFE après PCA, c'est qu'on perd l'interprétation heuristique qu'on peut faire sur l'importance d'un paramètre : l'œil humain ne peut plus lire des explications "naturelles".

Bien que la nécessité de faire du Preprocessing n'est plus à démontrer, il reste encore plusieurs indéterminations. Quels algorithmes faut-il mettre dans la pipeline ? Faut-il tous les prendre ou seulement certains parmi les 4 ? Ensuite, pour chaque algorithme gardé, il faut décider à quelle intensité on l'utilise en choisissant les paramètres comme le nombre de données frauduleuses à rajouter via SMOTE par exemple.

Enfin il est également possible de rendre ces algorithmes quantiques (par exemple QSMOTE [16]), c'est-à-dire agissant sur les vecteurs dans l'espace des états. Ou bien nous les utilisons pour optimiser notre base de données avant l'encodage quantique des vecteurs, ou bien nous les utilisons après l'encodage quantique, sur les vecteurs de notre FeatureMap. Le prétraitement post-mapping permet d'opérer directement sur la structure topologique de l'Espace de Hilbert, facilitant l'extraction de corrélations quantiques complexes qui resteraient cachées dans l'espace euclidien originel.

5 ALGORITHMES NON SUPERVISÉS

5.1 Clustering classique

5.2 Clustering quantique variationnel

Après avoir passé en revue les principales techniques de clustering quantique, nous avons retenu l'approche *variationnelle* (Variational Quantum Clustering, VQC) car c'est aujourd'hui la plus employée. Nous en reprenons directement la formulation proposée par Bermejo et Orús [13].

• Encodage et états de référence

On considère un jeu de N points $\mathcal{X} = \{\vec{x}_1, \ldots, \vec{x}_N\} \subset \mathbb{R}^d$ à partitionner en k clusters. On associe à chaque point $\vec{x}_i$ un vecteur de paramètres $\vec{\theta}_i \in [0, 2\pi)^n$, où n est le nombre de qubits dédiés à l'encodage. Dans sa forme la plus simple, l'encodage consiste en une séquence de rotations $R_Y(\theta_{i,j})$ appliquées aux n qubits initialement dans l'état $|0\rangle^{\otimes n}$:

$$|\psi(\vec{\theta}_i)\rangle \;=\; \bigotimes_{j=1}^{n} R_Y(\theta_{i,j})\,|0\rangle \;=\; \bigotimes_{j=1}^{n}\Big(\cos\tfrac{\theta_{i,j}}{2}\,|0\rangle + \sin\tfrac{\theta_{i,j}}{2}\,|1\rangle\Big)\,.$$

Ce schéma d'*angle encoding* est le plus léger en profondeur, donc le mieux adapté aux contraintes NISQ [22]. Des encodages plus expressifs — *ZZ feature map*, *amplitude encoding*, ou ansatz entanglant paramétré par couches $U_{\text{ent}}(\vec{\theta}_i)$ — peuvent s'y substituer sans modifier le reste de l'algorithme.

À chaque cluster $c \in \{1, \dots, k\}$ est associé un *état de référence* $|\phi_c\rangle \in (\mathbb{C}^2)^{\otimes n}$, tiré aléatoirement au début de l'exécution et *gelé* pendant tout l'apprentissage. Les paramètres entraînables sont donc uniquement les $\{\vec{\theta}_i\}_{i=1}^N$, qui encodent la position de chaque point dans l'espace de Hilbert relativement aux ancres fixes $\{|\phi_c\rangle\}_{c=1}^k$. On notera que les $|\phi_c\rangle$ ne forment pas nécessairement une base orthonormée : ils sont tirés indépendamment et peuvent avoir des recouvrements non nuls entre eux, ce qui est sans incidence sur la validité de l'algorithme.

• Matrice de fidélité et SWAP test

Le cœur du calcul est la *matrice de fidélité* $\mathcal{F} \in \mathbb{R}^{N \times k}$, définie par

$$\mathcal{F}_{ic} \;=\; \left|\langle \psi(\vec{\theta}_i) \,|\, \phi_c \rangle\right|^2.$$

On interprète $\mathcal{F}_{ic}$ comme un score continu d'appartenance du point i au cluster c : il vaut 1 si $|\psi(\vec{\theta}_i)\rangle = |\phi_c\rangle$ (à phase globale près), et 0 si les deux états sont orthogonaux. Contrairement à un k-means classique, qui produit une assignation dure, la formulation variationnelle manipule ici une *distribution* d'appartenance que l'on contraint ensuite à se concentrer sur un unique cluster via un terme de pénalité (cf. ci-dessous).

L'estimation effective de $\mathcal{F}_{ic}$ sur matériel quantique repose sur le *SWAP test*, qui extrait la fidélité entre deux états sans nécessiter de tomographie complète. On prépare un qubit ancillaire A dans $|+\rangle = \frac{1}{\sqrt{2}}(|0\rangle + |1\rangle)$, on applique des portes contrôlées `CSWAP` entre A et chaque paire de qubits correspondants de $|\psi(\vec{\theta}_i)\rangle$ et $|\phi_c\rangle$, puis on ramène A dans la base Z par une seconde Hadamard et on le mesure. On montre alors que la probabilité d'observer $A = 0$ vaut

$$\mathbb{P}(A = 0) \;=\; \tfrac{1}{2}\Big(1 + \left|\langle \psi(\vec{\theta}_i) \,|\, \phi_c \rangle\right|^2\Big),$$

d'où l'estimateur sans biais $\widehat{\mathcal{F}}_{ic} = 2\hat{p}_0 - 1$, où $\hat{p}_0$ est la fréquence empirique observée sur P répétitions. L'erreur quadratique de l'estimation décroît en $\mathcal{O}(1/P)$.

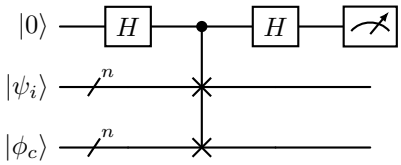

Figure 1 – Circuit du *SWAP test* estimant la fidélité $|\langle\psi_i|\phi_c\rangle|^2$. La ligne du haut est le qubit ancillaire ; les lignes inférieures, annotées n, représentent les deux registres de n qubits portant respectivement l'encodage du point $|\psi_i\rangle = |\psi(\vec{\theta}_i)\rangle$ et l'état de référence $|\phi_c\rangle$. La porte `CSWAP` échange ces deux registres qubit à qubit, conditionnée à l'ancilla. La mesure finale de l'ancilla fournit un estimateur non biaisé de la fidélité.

• Fonction de coût

Soit $D \in \mathbb{R}^{N \times N}$ la matrice des distances euclidiennes $D_{ij} = \|\vec{x}_i - \vec{x}_j\|_2$, calculée *une fois pour toutes* en prétraitement classique. La fonction de coût variationnelle s'écrit comme une combinaison linéaire de deux termes :

$$\mathcal{C}(\vec{\theta}) \;=\; \underbrace{\sum_{c=1}^{k} \sum_{i,j=1}^{N} D_{ij}\, \mathcal{F}_{ic}\, \mathcal{F}_{jc}}_{\mathcal{C}_{\text{cluster}}(\vec{\theta})} \;+\; \lambda \underbrace{\sum_{i=1}^{N} \Big(\sum_{c=1}^{k} \mathcal{F}_{ic} - 1 \Big)^2}_{\mathcal{C}_{\text{pénalité}}(\vec{\theta})}.$$

Le premier terme $\mathcal{C}_{\text{cluster}}$ pénalise l'appartenance simultanée de deux points *géométriquement éloignés* à un même cluster : il est d'autant plus grand que des paires (i, j) à D_{ij} élevée se voient attribuer des masses de fidélité $\mathcal{F}_{ic}\mathcal{F}_{jc}$ non négligeables sur un cluster commun. Sa minimisation encourage donc la compacité géométrique des groupes.

Le second terme $\mathcal{C}_{\text{pénalité}}$ impose que chaque point totalise une masse de fidélité proche de 1 sur l'ensemble des k clusters, empêchant la solution dégénérée $\mathcal{F}_{ic} \to 0$ qui annulerait trivialement $\mathcal{C}_{\text{cluster}}$ sans produire de

clustering. Il force ainsi, à la limite $\lambda \to \infty$, un comportement « one-hot » où chaque point se concentre sur un unique cluster.

Le scalaire $\lambda > 0$ règle l'équilibre entre ces deux contraintes et constitue l'unique hyperparamètre proprement variationnel du modèle. Un λ trop faible autorise des assignations floues sans incitation à clusteriser nettement ; un λ trop grand fait dominer la pénalité au point de masquer la géométrie sous-jacente. Les expériences du groupe précédent fixent $\lambda = 10$, valeur que nous retenons.

- **Algorithme complet**

L'algorithme complet assemble les ingrédients précédents en une boucle classique–quantique (figure 2) :

1. **Initialisation.** Normaliser les données, calculer une fois pour toutes la matrice D, tirer aléatoirement $\{\vec{\theta}_i\}_{i=1}^N \in [0, 2\pi)^{n \cdot N}$ et $\{|\phi_c\rangle\}_{c=1}^k$.
2. **Évaluation quantique.** Pour chaque couple (i, c), préparer $|\psi(\vec{\theta}_i)\rangle$ et $|\phi_c\rangle$, exécuter un SWAP test, et assembler la matrice $\mathcal{F}(\vec{\theta})$.
3. **Calcul classique du coût.** Évaluer $\mathcal{C}(\vec{\theta})$ à partir de D et $\mathcal{F}$.
4. **Mise à jour.** Utiliser un optimiseur classique (L-BFGS-B avec gradients numériques dans notre implémentation) pour mettre à jour $\vec{\theta}$, puis répéter les étapes 2–4 jusqu'à convergence.
5. **Assignation finale.** Pour chaque point i, retourner le cluster $c_i = \arg\max_c \mathcal{F}_{ic}$.

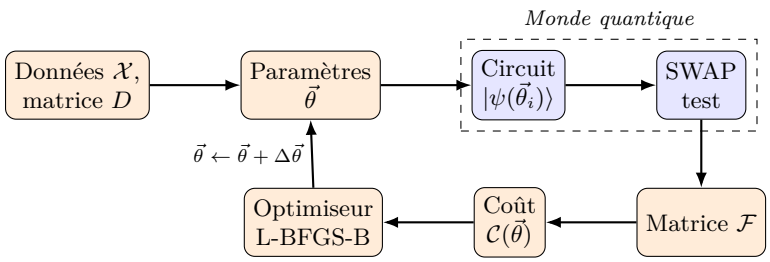

Figure 2 – Boucle d'apprentissage hybride du clustering variationnel quantique. Les blocs bleutés correspondent aux étapes exécutées sur simulateur/processeur quantique ; les blocs orange sont réalisés classiquement.

- **Choix du nombre de qubits et complexité**

Le nombre n de qubits dédiés à l'encodage joue un rôle crucial, mis empiriquement en évidence dans le rapport 2024-25. Deux régimes sont à éviter. En régime de *sous-paramétrisation* ($2^n < d$), la dimension de l'espace de Hilbert est trop faible pour distinguer les clusters et l'encodage projette des points distincts sur des états quasi confondus, induisant des fusions parasites. En régime de *sur-paramétrisation* ($2^n \gg d$), on effectue beaucoup de calculs inutiles et l'optimisation devient moins efficace. En pratique, choisir $n \approx \lceil \log_2 d \rceil$ offre le meilleur compromis.

La complexité par itération de l'optimiseur se décompose en deux contributions. Côté quantique, il faut $\mathcal{O}(Nk \cdot P)$ exécutions élémentaires du circuit SWAP test pour remplir la matrice de fidélité $\mathcal{F}$, où N est le nombre de points, k le nombre de clusters, et P le nombre de répétitions utilisées pour estimer chaque fidélité. Côté classique, l'évaluation de la fonction de coût $\mathcal{C}(\vec{\theta})$ à partir de $\mathcal{F}$ et D coûte $\mathcal{O}(N^2 k)$ opérations, en raison de la double somme sur les paires (i, j) dans le terme $\mathcal{C}_{\text{cluster}}$.

5.3 Implémentation

• Métrique d'évaluation : l'indice de Rand ajusté (ARI)

Pour évaluer la qualité du clustering, nous utilisons l'*indice de Rand ajusté* (ARI, *Adjusted Rand Index*), qui mesure la similarité entre la partition prédite $\hat{\mathcal{P}} = \{\hat{C}_1, \dots, \hat{C}_k\}$ et la partition de référence $\mathcal{P}^\star = \{C_1^\star, \dots, C_{k'}^\star\}$. En notant $n_{ij} = |\hat{C}_i \cap C_j^\star|$, $a_i = \sum_j n_{ij}$ et $b_j = \sum_i n_{ij}$,

$$\mathrm{ARI} = \frac{\sum_{i,j} \binom{n_{ij}}{2} - \frac{1}{\binom{N}{2}} \sum_i \binom{a_i}{2} \sum_j \binom{b_j}{2}}{\frac{1}{2}\left(\sum_i \binom{a_i}{2} + \sum_j \binom{b_j}{2}\right) - \frac{1}{\binom{N}{2}} \sum_i \binom{a_i}{2} \sum_j \binom{b_j}{2}}.$$

Le numérateur compte les paires de points regroupées à l'identique dans les deux partitions (terme $\sum_{i,j} \binom{n_{ij}}{2}$), auquel on retranche le nombre attendu de telles paires sous tirage aléatoire. Ainsi, ARI $= 1$ lorsque les deux partitions coïncident parfaitement et ARI $= 0$ lorsque la partition prédite n'est pas meilleure qu'un tirage au hasard.

• Première expérience

Nous avons implémenté l'algorithme VQC décrit plus haut en *Qiskit*. Le jeu de données synthétique est constitué de trois *blobs* gaussiens dans $\mathbb{R}^3$, chacun de cardinalité 20 (soit $N = 60$ points à partitionner en $k = 3$ clusters). Nous avons ensuite exécuté la boucle hybride décrite précédemment pendant 30 itérations de l'optimiseur, avec l'hyperparamètre $\lambda = 10$.

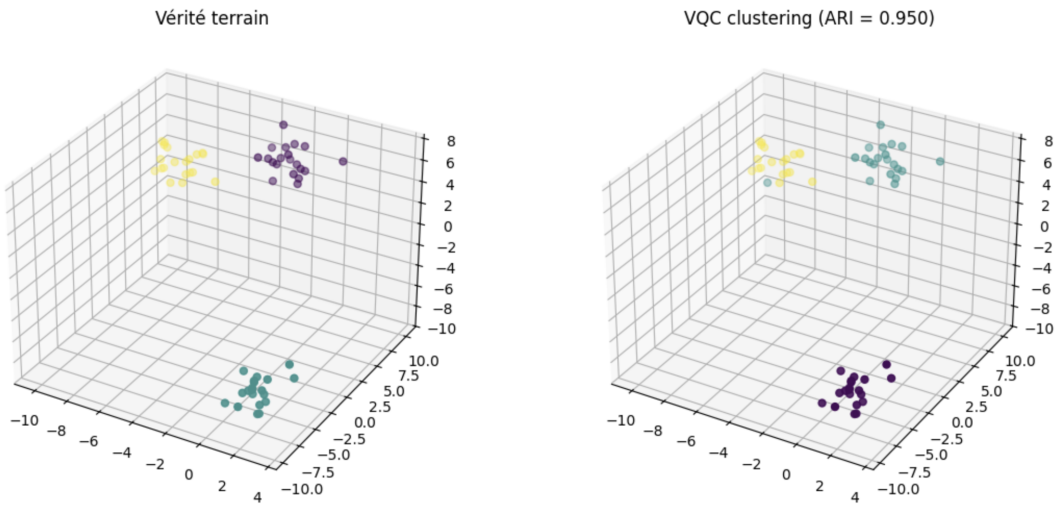

Figure 3 – Résultats de la première exécution du clustering variationnel quantique sur trois blobs gaussiens dans $\mathbb{R}^3$ ($N = 60$, $k = 3$, $\lambda = 10$, 30 itérations).

L'ARI obtenu sur cette exécution vaut 0,950, ce qui signifie que la VQC a bien compris la répartition en clusters. Ce résultat confirme que la boucle variationnelle parvient à exploiter la géométrie des blobs malgré le faible nombre d'itérations, et valide notre implémentation.

• Application au jeu de données de cartes bancaires

Après avoir validé l'implémentation sur des données synthétiques, nous avons appliqué le clustering variationnel quantique au jeu de données de cartes bancaires de Kaggle. Nous en tirons un sous-ensemble de $N = 200$ transactions, dont 10 fraudes avérées. L'algorithme est exécuté avec $n = 6$ qubits et l'hyperparamètre $\lambda = 10$.

Afin de visualiser la partition obtenue dans un espace de dimension raisonnable, nous projetons les données sur les trois composantes principales issues d'une ACP, puis affichons l'affectation des points aux clusters (figure 4).

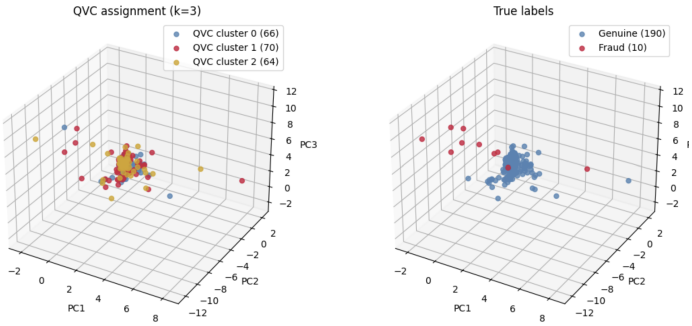

FIGURE 4 – Clustering quantique variationnel sur 200 transactions (10 fraudes, $n = 6$ qubits, $\lambda = 10$), visualisé dans l'espace des trois composantes principales.

Pour détecter les fraudes à partir de cette partition, nous exploitons l'incertitude d'affectation du VQC. Pour chaque transaction i, nous disposons des probabilités d'appartenance F_{ic} à chacun des clusters c et définissons le score de confiance $s_i = \max_c F_{ic}$. Nous fixons alors un seuil τ tel qu'une transaction vérifiant $s_i < \tau$ soit considérée comme mal affectée et étiquetée comme fraude. Le seuil optimal $\tau^\star$ est obtenu en itérant sur l'ensemble des valeurs admissibles et en retenant celui qui maximise le score F_1 sur les 10 fraudes connues. La courbe d'évolution du F_1 en fonction du seuil est reportée sur la figure 5 et les résultats optimaux sont rassemblés dans le tableau 1.

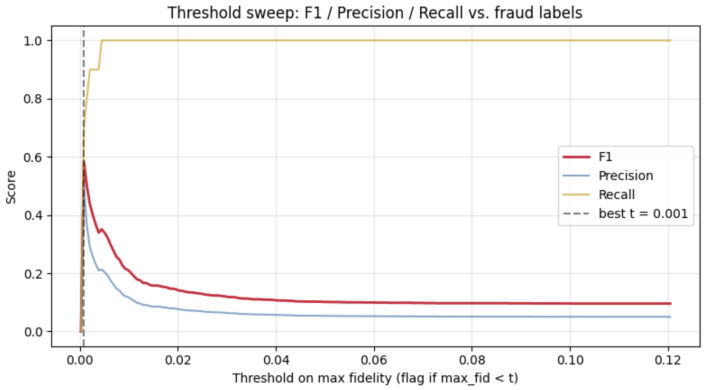

FIGURE 5 – Évolution du score F_1 du VQC en fonction du seuil de confiance τ.

Métrique	Valeur
Seuil optimal $\tau^\star$	0,0009
F_1	0,5833
Précision	0,5000 (7 TP / 14 signalés)
Rappel	0,7000 (7 TP / 10 fraudes)
Transactions signalées	14/200

TABLE 1 – Résultats optimaux du clustering quantique variationnel sur le sous-ensemble de 200 transactions.

• Benchmark avec l'algorithme k-means

Nous avons ensuite appliqué la même procédure au clustering classique des k-means sur le même sous-ensemble, afin de disposer d'un point de comparaison. La stratégie de détection diffère légèrement : plutôt que d'utiliser une probabilité d'affectation, nous exploitons la distance euclidienne d_i de chaque transaction au centroïde de son cluster d'affectation. Une transaction est signalée comme fraude si $d_i > \tau$, et le seuil τ est à nouveau choisi pour maximiser le F_1. Les clusters obtenus sur les trois premières composantes principales sont présentés sur la figure 6, la courbe d'évolution du F_1 sur la figure 7 et les résultats optimaux dans le tableau 2.

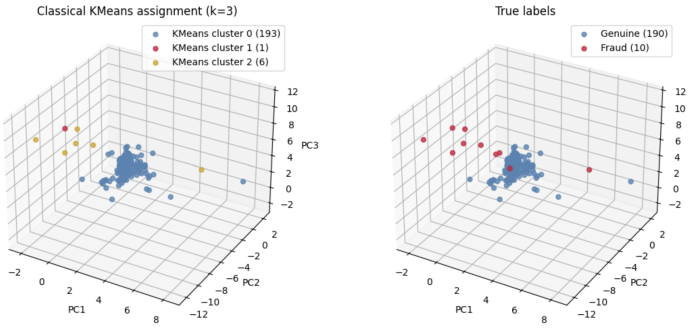

Figure 6 – Clustering k-means sur les mêmes 200 transactions, visualisé dans l'espace des trois composantes principales.

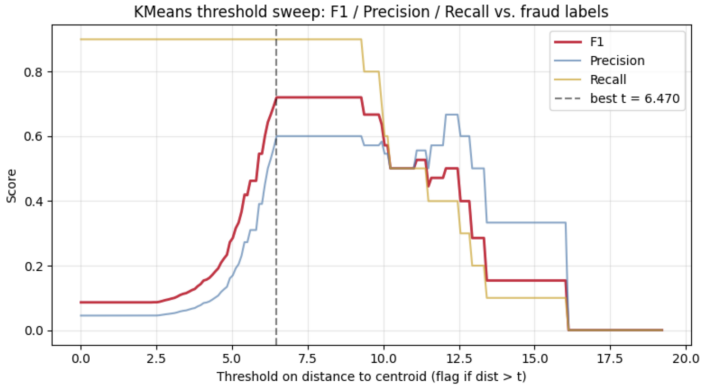

Figure 7 – Évolution du score F_1 du k-means en fonction du seuil de distance au centroïde.

Métrique	Valeur
Seuil optimal $\tau^\star$	6,4702
F_1	0,7200
Précision	0,6000 (9 TP / 15 signalés)
Rappel	0,9000 (9 TP / 10 fraudes)
Transactions signalées	15/200

Table 2 – Résultats optimaux du k-means sur le même sous-ensemble de 200 transactions.

Interprétation. De prime abord, le k-means demeure plus efficace que le clustering quantique variationnel, avec un F_1 supérieur (0,72 contre 0,58) et des clusters visuellement plus nets. L'écart se creuse en particulier sur le *rappel* (0,90 contre 0,70), ce qui est d'autant plus significatif que cette métrique est la plus critique en

détection de fraude : manquer une transaction frauduleuse (faux négatif) coûte bien plus cher à la banque qu'une alerte injustifiée. Une explication plausible de ce déficit réside dans la simplicité des transformations considérées dans notre circuit d'encodage : la *feature map* utilisée ne produit pas un plongement suffisamment riche pour que les fraudes se distinguent clairement des transactions légitimes via les probabilités F_{ic}. Il apparaît donc nécessaire d'améliorer cette approche — par exemple en enrichissant la *feature map* ou en repensant le critère de détection — ce que nous explorons dans le paragraphe suivant, avant de nous tourner, dans la section suivante, vers un algorithme quantique plus spécifiquement conçu pour la détection d'anomalies, le Q-OCSVM.

• Clustering quantique variationnel amélioré

Un examen critique de l'implémentation de base révèle trois limitations structurelles qui bornent ce que le modèle peut effectivement apprendre.

1. **Absence d'intrication.** Le circuit d'encodage se réduit à un produit tensoriel de blocs $[\cos(\theta/2), \sin(\theta/2)]$ qubit par qubit (uniquement des portes R_Y, sans CNOT ni CZ). L'ensemble des états atteignables est donc un tore de dimension n_{qubits} plongé dans l'espace de Hilbert de dimension $2^{n_{\text{qubits}}}$: nous restons confinés aux états produits classiques et nous n'exploitons aucune corrélation quantique.
2. **Amplitudes exclusivement réelles.** Les portes utilisées n'engendrent que des amplitudes réelles (cos et sin d'angles réels), alors que les références cibles tirées uniformément dans $\mathbb{C}^{2^n}$ possèdent une partie imaginaire. Les fidélités $|\langle\phi|\psi\rangle|^2$ ne retiennent que la composante réelle et un grand nombre de références distinctes deviennent indiscernables sur cette variété.
3. **Pénalité de normalisation mal posée.** La contrainte $\sum_c \mathcal{F}_{ic} \to 1$ n'est valide que si les $\{|\phi_c\rangle\}$ forment une base orthonormée complète ($k = d$). Or, avec $k = 3$ vecteurs tirés dans $d = 2^5 = 32$ dimensions, l'espérance de cette somme vaut k/d, et non 1 ; la pénalité pousse l'optimiseur vers une cible inatteignable.

Nous reconstruisons le modèle de façon à lever simultanément ces trois obstacles, tout en conservant la méthode de détection par balayage de seuil sur $s_i = \max_c \mathcal{F}_{ic}$ de la section précédente.

Rotations à un qubit. Les portes $R_X(\theta)$, $R_Y(\theta)$ et $R_Z(\theta)$ désignent les rotations d'un qubit d'un angle θ autour des axes x, y et z de la sphère de Bloch, définies par $R_\sigma(\theta) = \exp(-i\,\theta\,\sigma/2)$ où $\sigma \in \{X, Y, Z\}$ est la matrice de Pauli correspondante. Matriciellement,

$$R_Y(\theta) = \begin{pmatrix} \cos\frac{\theta}{2} & -\sin\frac{\theta}{2} \\ \sin\frac{\theta}{2} & \cos\frac{\theta}{2} \end{pmatrix}, \quad R_Z(\theta) = \begin{pmatrix} e^{-i\theta/2} & 0 \\ 0 & e^{i\theta/2} \end{pmatrix}, \quad R_X(\theta) = \begin{pmatrix} \cos\frac{\theta}{2} & -i\sin\frac{\theta}{2} \\ -i\sin\frac{\theta}{2} & \cos\frac{\theta}{2} \end{pmatrix}.$$

En particulier, R_Y appliquée à $|0\rangle$ n'engendre que des amplitudes réelles, ce qui correspond à la limitation (2) identifiée plus haut et justifie l'introduction de R_Z (qui ajoute des phases complexes) dans le bloc variationnel.

Modifications apportées.

— *Feature map avec intrication.* Nous appliquons $R_Y(x)$ sur chaque qubit, puis un anneau de portes CZ, puis $R_Y(2x)$ sur chaque qubit. À partir de $|0\rangle^{\otimes n}$, les caractéristiques (coordonnées ACP des transactions) produisent désormais un état véritablement intriqué, dont les amplitudes dépendent non linéairement des entrées.

— *Bloc variationnel partagé.* Nous ajoutons n_{layers} couches composées, pour chaque qubit, d'une séquence R_Y–R_Z–R_Y suivie d'un anneau CZ. Les angles entraînables θ, de forme $(n_{\text{layers}}, n_{\text{qubits}}, 3)$, sont partagés entre tous les échantillons, ce qui ramène le nombre de paramètres à quelques dizaines (au lieu de $N \cdot n$) et rend la descente *L-BFGS-B* à différences finies peu coûteuse.

— *États de cluster libres.* Chaque $|\phi_c\rangle$ est un vecteur complexe plein de $\mathbb{C}^{2^{n_{\text{qubits}}}}$, renormalisé à l'usage. Les références ne sont plus contraintes à la sous-variété des états produits.

— *Fonction de coût corrigée.*

$$\mathcal{C}(\theta, \Phi) \;=\; \underbrace{\tfrac{1}{N}\sum_{i=1}^{N}\big(1 - \operatorname{smax}_\tau(\mathcal{F}_{i\cdot})\big)^2}_{\text{couverture}} \;+\; \tfrac{1}{2}\,\underbrace{\tfrac{1}{k^2}\left\|\Phi\Phi^\dagger - I_k\right\|_F^2}_{\text{séparation}}$$

où smax_τ est un substitut lisse de $\max_c \mathcal{F}_{ic}$. Le terme de *couverture* pousse chaque échantillon à s'affecter franchement à un cluster ; le terme de *séparation* remplace la pénalité mal posée $\sum_c \mathcal{F}_{ic} = 1$ par la contrainte que les états de cluster soient deux à deux orthonormés ($\Phi\Phi^\dagger \approx I_k$), condition sous laquelle les fidélités somment effectivement à 1.

— *Tête de détection inchangée.* Chaque point reçoit le score $1 - \max_c \mathcal{F}_{ic}$; on balaye ensuite le seuil de coupure et on retient celui qui maximise le F_1, exactement comme pour le VQC de base et le k-means.

Nous appliquons ce modèle amélioré au même sous-ensemble de 200 transactions (dont 10 fraudes), avec les mêmes hyperparamètres ($n = 6$ qubits, projection ACP identique). La figure 8 montre l'affectation des points aux clusters sur les trois composantes principales, la figure 9 la courbe d'évolution du F_1 en fonction du seuil, et le tableau 3 récapitule les scores au seuil optimal.

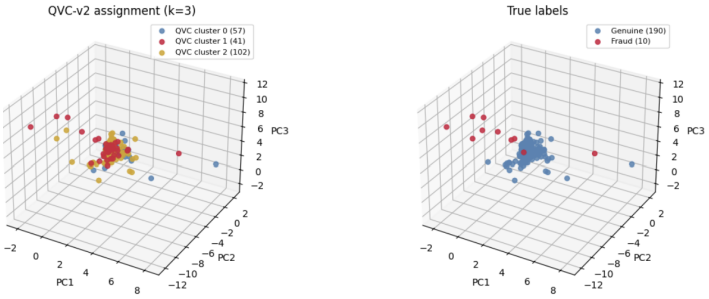

FIGURE 8 – Clustering quantique variationnel *amélioré* sur les mêmes 200 transactions, visualisé dans l'espace des trois composantes principales.

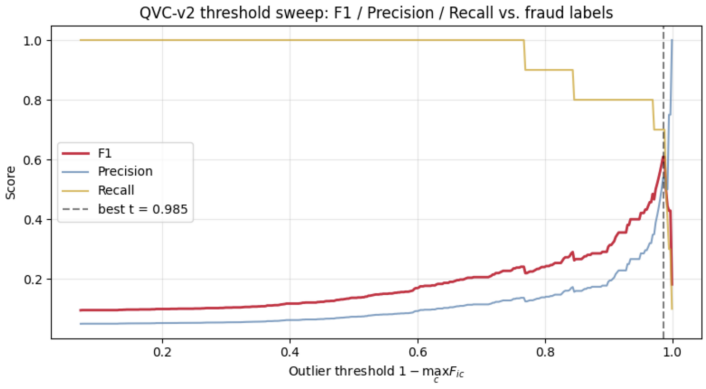

FIGURE 9 – Évolution du score F_1 du VQC amélioré en fonction du seuil de confiance τ.

Métrique	Valeur
Seuil optimal $\tau^\star$	0,9853
F_1	0,6087
Précision	0,5385 (7 TP / 13 signalés)
Rappel	0,7000 (7 TP / 10 fraudes)
Transactions signalées	13/200

TABLE 3 – Résultats optimaux du clustering quantique variationnel *amélioré* sur le sous-ensemble de 200 transactions.

Bilan. Cette version corrigée atteint $F_1 = 0{,}61$ (contre 0,58 pour le VQC de base), avec un rappel identique (0,70) mais une précision légèrement améliorée (0,538 contre 0,500). Le gain, réel mais modeste, confirme que l'intrication, les amplitudes complexes et la contrainte d'orthonormalité apportent une expressivité supplémentaire ; il reste toutefois insuffisant pour combler l'écart avec le k-means sur ce jeu, ce qui nous a amené à pousser l'approche encore plus loin dans une dernière version décrite ci-après.

• Clustering quantique variationnel v3

Enfin, nous présentons une dernière amélioration du clustering quantique, qui s'appuie sur la version précédente en lui ajoutant deux ingrédients : un terme de coût qui préserve la structure locale des données, et un entraînement robuste empêchant les fraudes de contaminer le signal d'apprentissage.

Changements par rapport à la version améliorée.

— *Préservation des similarités.* Nous ajoutons au coût un terme $\mathcal{C}_{\text{sim}} = \frac{1}{|E|}\sum_{(i,j)\in E}\big(1-|\langle\psi_i|\psi_j\rangle|^2\big)^2$, sommé sur les arêtes E d'un graphe des $k_{\text{NN}} = 5$ plus proches voisins calculé sur les données pré-encodées. Ce terme force les points proches dans l'espace initial à le rester dans l'espace de Hilbert, ce qui injecte explicitement l'information de distance qui n'entrait jusqu'ici qu'implicitement via la continuité du plongement.

— *Entraînement robuste par élagage.* Avant l'optimisation, nous appliquons un k-means classique sur les données pré-encodées et écartons les 5 % de points les plus éloignés de leur centroïde. Ces points, supposés être des fraudes en puissance, sont exclus à la fois du terme de couverture et du graphe E. Sans cet élagage, la couverture inciterait l'optimiseur à aspirer les fraudes dans les clusters et ferait s'effondrer la capacité de détection ; avec lui, les ancres se calent sur la masse saine. Les points écartés sont en revanche toujours scorés à l'inférence.

— *Tête de détection inchangée.* On conserve $s_i = 1 - \max_c \mathcal{F}_{ic}$ et le balayage du seuil maximisant le F_1.

Nous appliquons ce modèle au même sous-ensemble de 200 transactions. Les résultats sont rassemblés sur la figure 10 (affectations, étiquettes réelles et courbe F_1 vs seuil) et dans le tableau 4.

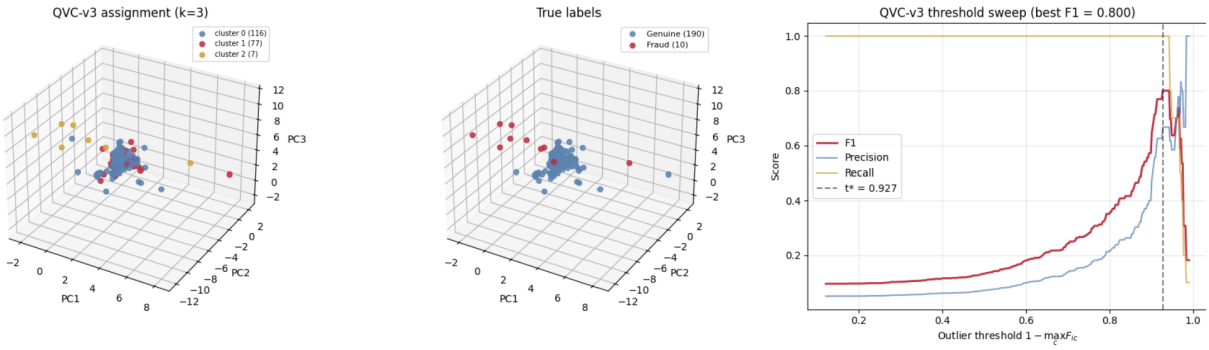

Figure 10 – VQC v3 : affectation aux $k = 3$ clusters (gauche), étiquettes réelles (centre), et évolution du F_1 en fonction du seuil de détection τ (droite). Les points sont projetés sur les trois premières composantes ACP.

Métrique	Valeur
Seuil optimal $\tau^\star$	0,9275
F_1	0,8000
Précision	0,6667 (10 TP / 15 signalés)
Rappel	1,0000 (10 TP / 10 fraudes)
Transactions signalées	15/200

Table 4 – Résultats optimaux du clustering quantique variationnel *v3* sur le sous-ensemble de 200 transactions.

«««< HEAD

Bilan. Cette version atteint $F_1 = 0{,}80$, avec un rappel *parfait* (1,00, soit 10/10 fraudes détectées) pour 15 transactions signalées sur 200, et dépasse ainsi le k-means ($F_1 = 0{,}72$, rappel 0,90). Ce gain est en grande partie imputable au *trimming*, qui empêche les fraudes de contaminer l'entraînement, et à la préservation des similarités, qui donne au plongement une géométrie plus cohérente. Nous poursuivons maintenant avec l'examen du QSVM.

=======

Bilan. Cette version atteint $F_1 = 0{,}80$, avec un rappel *parfait* (1,00, soit 10/10 fraudes détectées) pour 15 transactions signalées sur 200, et dépasse ainsi le k-means ($F_1 = 0{,}72$, rappel 0,90). Ce gain est en grande partie imputable au *trimming*, qui empêche les fraudes de contaminer l'entraînement, et à la préservation des similarités, qui donne au plongement une géométrie plus cohérente. Nous poursuivons maintenant avec l'examen du Q-OCSVM »»»> 610c7c94086f27e24572c8933e5219a773353c39

5.4 Rappels théoriques : SVM classique et astuce du noyau

Avant d'aborder sa variante quantique, il est indispensable de poser le cadre mathématique du Support Vector Machine (SVM) classique, car l'approche quantique vient s'y greffer directement sans en altérer la logique d'optimisation.

À l'origine, le SVM est un modèle de classification binaire cherchant à séparer linéairement des données réparties en deux classes [25]. L'idée directrice n'est pas seulement de trouver un hyperplan séparateur d'équation $\vec{w} \cdot \vec{x} + b = 0$, mais de trouver celui qui maximise la marge, c'est-à-dire la distance entre l'hyperplan et les points les plus proches de chaque classe.

• Le problème primal et la marge souple

Mathématiquement, maximiser cette marge revient à minimiser la norme du vecteur normal à l'hyperplan, $\|\vec{w}\|$. Or, dans le cas de données réelles (et a fortiori dans la détection de fraude), les classes se chevauchent souvent à cause du bruit. Il est donc impossible de tracer une frontière parfaite. Pour y remédier, nous introduisons des variables réelles $\xi_i \geq 0$ qui autorisent certaines erreurs de classification moyennant une pénalité.

Le problème d'optimisation, dit « primal », s'écrit alors :

$$\min_{\vec{w},b,\vec{\xi}} \frac{1}{2}\|\vec{w}\|^2 + C\sum_{i=1}^{N} \xi_i$$

sous les contraintes $y_i(\vec{w} \cdot \vec{x}_i + b) \geq 1 - \xi_i$. Le paramètre de régularisation C contrôle le compromis entre la largeur de la marge (généralisation) et la pénalisation des erreurs sur le jeu d'entraînement. $y_i \in \{-1, 1\}$ est le label du point x_i. $y_i(\vec{w} \cdot \vec{x}_i + b)$ est censé être supérieur ou égal à 1, ainsi ξ_i est la variable de pénalisation de cette contrainte qu'on se permet d'assouplir.

• Passage au problème dual

La formulation primale dépend directement de la dimensionnalité des données. Si nous voulons projeter nos données dans un espace très complexe pour capturer des comportements non linéaires, la taille de $\vec{w}$ explosera, rendant le calcul intraitable.

La solution canonique consiste à passer par la formulation duale de ce problème d'optimisation en introduisant les multiplicateurs de Lagrange α_i. Après dérivation et annulation du gradient par rapport à $\vec{w}$ et b, le problème devient la maximisation d'une fonction quadratique :

$$\max_{\vec{\alpha}} \sum_{i=1}^{N} \alpha_i - \frac{1}{2}\sum_{i=1}^{N}\sum_{j=1}^{N} \alpha_i \alpha_j y_i y_j (\vec{x}_i \cdot \vec{x}_j)$$

sous les contraintes $0 \leq \alpha_i \leq C$ et $\sum_i \alpha_i y_i = 0$.

Ce problème présente deux propriétés fondamentales pour la suite de notre architecture :

1. **Parcimonie des solutions :** La majorité des multiplicateurs α_i s'annulent. Seuls les points d'entraînement situés sur la bordure de la marge ou du mauvais côté auront un $\alpha_i > 0$. Ces points critiques sont appelés les *vecteurs de support*. La frontière de décision finale ne dépend plus que d'eux.
2. **Dépendance exclusive au produit scalaire :** Les vecteurs de données $\vec{x}$ n'apparaissent dans la fonction à optimiser que sous la forme de produits scalaires $(\vec{x}_i \cdot \vec{x}_j)$, ce qui nous permet d'utiliser l'astuce du noyau ou *kernel trick* en anglais.

- **Le Kernel Trick (Astuce du noyau)**

S'il est trop optimiste de pouvoir tracer une frontière rectiligne pertinente dans l'espace de départ qui est de faible dimension, nous pouvons transformer nos données via une fonction non linéaire $\phi : \mathbb{R}^d \to \mathcal{F}$, où $\mathcal{F}$ est un espace de bien plus grande dimension.

Plutôt que de calculer explicitement les lourdes coordonnées de $\phi(\vec{x}_i)$ — ce qui serait coûteux voire impossible si $\mathcal{F}$ est de dimension infinie —, la littérature classique [26] souligne que l'algorithme dual n'a besoin que du produit scalaire dans ce nouvel espace. Il suffit alors de définir une fonction noyau K telle que :

$$K(\vec{x}_i, \vec{x}_j) \;=\; \phi(\vec{x}_i) \cdot \phi(\vec{x}_j)$$

On remplace simplement le terme $(\vec{x}_i \cdot \vec{x}_j)$ par $K(\vec{x}_i, \vec{x}_j)$ dans le problème dual. L'algorithme se comporte alors comme si nous avions optimisé un hyperplan dans un espace très complexe, celui où vivent les points $\phi(\vec{x}_i)$, tout en conservant une complexité de calcul parfaitement maîtrisée. C'est précisément cette matrice des similarités K (matrice de Gram) que l'ordinateur quantique va se charger de calculer pour nous dans l'approche hybride.

5.5 Quantum One-Class Support Vector Machine (Q-OCSVM)

Si le clustering variationnel vise à regrouper les données en plusieurs catégories, il est souvent plus naturel, dans le contexte de la fraude bancaire, de ne chercher qu'à modéliser ce qui est « normal ». La fraude étant par nature rare et multiforme, il est difficile d'en faire un cluster à part entière. Le *Quantum One-Class SVM* (Q-OCSVM) répond exactement à cette logique, en adaptant au monde quantique le concept d'estimation de support [27] : au lieu de partitionner l'espace, nous cherchons à tracer une frontière la plus ajustée possible autour des transactions légitimes. Le problème d'optimisation primal s'écrit alors :

$$\begin{aligned} \min_{\vec{w},\rho,\vec{\xi}} \quad & \frac{1}{2}\|\vec{w}\|^2 - \rho + \frac{1}{\nu N}\sum_{i=1}^{N} \xi_i \\ \text{sous contraintes} \quad & \vec{w} \cdot \phi(\vec{x}_i) \geq \rho - \xi_i, \quad \forall i \in \{1, \ldots, N\} \\ & \xi_i \geq 0, \quad \forall i \in \{1, \ldots, N\} \end{aligned} \tag{1}$$

En introduisant les multiplicateurs de Lagrange α_i associés aux contraintes de marge, le passage au dual (cf. section 5.4) fait disparaître $\vec{w}$, ρ et $\vec{\xi}$, et ne laisse apparaître les données qu'au travers du noyau $K(\vec{x}_i, \vec{x}_j) = \langle \phi(\vec{x}_i) \,|\, \phi(\vec{x}_j) \rangle$. Le problème dual s'écrit :

$$\begin{aligned} \min_{\vec{\alpha}} \quad & \frac{1}{2}\sum_{i,j=1}^{N} \alpha_i \alpha_j \, K(\vec{x}_i, \vec{x}_j) \\ \text{sous contraintes} \quad & 0 \leq \alpha_i \leq \frac{1}{\nu N}, \quad \forall i \in \{1, \ldots, N\} \\ & \sum_{i=1}^{N} \alpha_i = 1 \end{aligned} \tag{2}$$

C'est cette formulation duale, dont la complexité ne dépend que de N et qui n'interagit avec les données qu'à travers la matrice de Gram K, qui permet précisément de brancher un calcul de noyau quantique à la place du noyau classique, sans rien changer à l'étape d'optimisation.

Ainsi, bien que le SVM classique soit fondamentalement un algorithme d'apprentissage supervisé (nécessitant des étiquettes pour séparer deux classes distinctes), l'OCSVM s'inscrit dans un cadre strictement non supervisé. L'astuce géométrique consiste à fournir uniquement des données normales à l'algorithme (ou à défaut des données passées) et à contraindre l'optimisation pour que tous ces points soient confinés du même côté d'un hyperplan, tandis que l'origine de l'espace de représentation est artificiellement repoussée de l'autre côté, jouant le rôle de la classe adverse.

L'intérêt majeur de cette méthode pour notre pipeline hybride réside dans la séparation très nette des tâches. Nous n'utilisons le processeur quantique que pour une seule chose : évaluer à quel point deux transactions se ressemblent. Toute la partie d'optimisation (le tracé de la frontière) reste confiée à un algorithme classique.

• Plongement quantique et matrice de Gram

Classiquement, les algorithmes comme le SVM utilisent l'astuce du noyau (*kernel trick*) pour projeter les données dans un espace de plus grande dimension (via un noyau gaussien RBF par exemple) afin de faciliter la séparation des points. Dans notre approche quantique, c'est le circuit lui-même qui sert de fonction de projection.

Concrètement, chaque transaction $\vec{x}_i \in \mathbb{R}^d$ est encodée sous la forme d'un état quantique à l'aide d'un circuit paramétré $U(\vec{x}_i)$. L'état obtenu est :

$$|\psi(\vec{x}_i)\rangle \;=\; U(\vec{x}_i)\,|0\rangle^{\otimes n}$$

L'espace dans lequel nous travaillons n'est plus un espace euclidien standard, mais l'espace de Hilbert de nos n qubits, dont la dimension croît de manière exponentielle en 2^n. En utilisant des circuits d'encodage qui intriquent les qubits (comme les feature maps basés sur des portes de Pauli [10]), nous exploitons des corrélations très complexes entre les différentes caractéristiques (montant, heure, localisation) de la transaction. La similarité entre deux transactions $\vec{x}_i$ et $\vec{x}_j$ s'écrit alors comme le produit scalaire de leurs deux états quantiques :

$$K_Q(\vec{x}_i, \vec{x}_j) \;=\; \langle \psi(\vec{x}_i) \,|\, \psi(\vec{x}_j)\rangle$$

(avec des circuits à amplitudes réelles, ce qui est notre cas, K_Q est réel et positif).

• Le circuit de calcul (Compute-Uncompute)

Pour estimer la similarité entre deux états sur machine, nous pourrions utiliser le *SWAP test* présenté dans la section précédente. Cependant, le SWAP test nécessite de doubler le nombre de qubits pour stocker les deux états simultanément, en plus d'un qubit ancillaire. Sur les processeurs actuels, chaque qubit supplémentaire apporte du bruit.

D'après la littérature d'apprentissage quantique à base de noyaux [28] il est préférable d'utiliser une technique appelée *inversion* ou *compute-uncompute*. L'idée est de n'utiliser qu'un seul registre de n qubits. On applique d'abord le circuit d'encodage pour la première transaction, puis on applique l'inverse du circuit paramétré avec la seconde transaction. L'état final du registre s'écrit :

$$|\psi_{i,j}\rangle \;=\; U^\dagger(\vec{x}_j)\,U(\vec{x}_i)\,|0\rangle^{\otimes n}\,.$$

La probabilité de mesurer la chaîne $|0\rangle^{\otimes n}$ dans la base computationnelle vaut alors, par définition :

$$P\big(0^{\otimes n}\big) \;=\; \big|\langle 0|^{\otimes n}\,U^\dagger(\vec{x}_j)\,U(\vec{x}_i)\,|0\rangle^{\otimes n}\big|^2 \;=\; \big|\langle \psi(\vec{x}_j) \,|\, \psi(\vec{x}_i)\rangle\big|^2 \;=\; \big|K_Q(\vec{x}_i, \vec{x}_j)\big|^2.$$

Protocole d'estimation de $K_Q(\vec{x}_i, \vec{x}_j)$.

1. Préparer le registre dans l'état $|0\rangle^{\otimes n}$.
2. Appliquer le circuit d'encodage $U(\vec{x}_i)$ (*compute*).
3. Appliquer son inverse $U^\dagger(\vec{x}_j)$ paramétré par le second point (*uncompute*).

4. Mesurer tous les qubits dans la base computationnelle.
5. Répéter P fois et estimer $\widehat{|K_Q|^2} = n_0/P$, où n_0 est le nombre de tirs ayant produit la chaîne $00\ldots0$, puis poser $\widehat{K}_Q(\vec{x}_i, \vec{x}_j) = \sqrt{\widehat{|K_Q|^2}}$.

L'estimateur du module au carré converge vers $|K_Q|^2$ avec une variance en $\mathcal{O}(1/P)$; le nombre de tirs fixe donc la précision du noyau transmis au solveur classique.

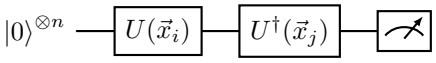

FIGURE 11 – Évaluation de la similarité entre deux transactions par la méthode d'inversion. Le registre est préparé selon les caractéristiques du point $\vec{x}_i$, puis subit la transformation inverse paramétrée par $\vec{x}_j$. La fréquence de mesure de l'état fondamental $|0\ldots0\rangle$ nous donne le module au carré du noyau $|K_Q(\vec{x}_i, \vec{x}_j)|^2$.

• Entraînement et limites de passage à l'échelle

Une fois que le circuit quantique a calculé les similarités pour toutes les paires de transactions de notre jeu d'entraînement, nous obtenons une matrice dense, la matrice de Gram. Celle-ci est transmise à un optimiseur classique qui va chercher à isoler les données en ajustant un paramètre ν qui est inversement proportionnel au paramètre de pénalité C indiqué dans l'équation d'optimisation du SVM dans la section précédente (et qui représente concrètement la proportion de faux positifs que la banque est prête à tolérer).

Le principal verrou de cette méthode, que nous avons dû affronter lors des tests sur les données de la BNP, est sa complexité quadratique. Pour N transactions d'entraînement, il faut exécuter $N(N-1)/2$ circuits quantiques, et ce, pour plusieurs milliers de batchs à chaque fois afin d'avoir une estimation statistique fiable. Sur un jeu de données bancaire massif, le calcul complet de la matrice devient vite irréalisable en un temps raisonnable. C'est pourquoi nous avons dû accepter l'idée de ne conserver qu'un sous-échantillon lors des implémentations de nos algorithmes sur le dataset fourni par la BNP.

• Algorithme complet

L'implémentation de notre brique Q-OCSVM suit la séquence classique-quantique suivante :

1. **Préparation classique :** À partir de l'ensemble d'entraînement constitué de transactions supposées normales $\mathcal{X}_{\text{norm}} \subset \mathbb{R}^d$, on sélectionne (par sous-échantillonnage) un sous-ensemble gérable de N transactions.
2. **Évaluation quantique (Construction de la matrice de Gram) :**
   — Initialiser une matrice $\mathcal{K}$ de taille $N \times N$.
   — Pour chaque paire de points $(\vec{x}_i, \vec{x}_j)$ avec $i \leq j$:
     — Exécuter le circuit d'inversion $U^\dagger(\vec{x}_j)U(\vec{x}_i)\,|0\rangle^{\otimes n}$ sur le simulateur.
     — Estimer le module au carré $|K_Q(\vec{x}_i, \vec{x}_j)|^2$ via la fréquence de mesure de l'état $|0\rangle^{\otimes n}$ sur P répétitions, puis en prendre la racine carrée pour obtenir $K_Q(\vec{x}_i, \vec{x}_j)$ (circuit à amplitudes réelles positives).
     — Remplir la matrice : $\mathcal{K}_{ij} = \mathcal{K}_{ji} = K_Q(\vec{x}_i, \vec{x}_j)$.
3. **Optimisation classique :** Fournir la matrice de Gram $\mathcal{K}$ et le paramètre ν à un solveur SVM classique (par exemple via `scikit-learn`) pour déterminer les vecteurs de support $\vec{\alpha}$ et le biais ρ.
4. **Inférence (Nouvelle transaction) :** Pour évaluer une nouvelle transaction $\vec{z}$:
   — Utiliser le circuit quantique pour calculer les noyaux $K_Q(\vec{x}_{\text{sv}}, \vec{z})$ uniquement avec les transactions $\vec{x}_{\text{sv}}$ identifiées comme vecteurs de support lors de l'entraînement.
   — Calculer classiquement le score de décision $f(\vec{z}) = \sum_{\text{sv}} \alpha_{\text{sv}} K_Q(\vec{x}_{\text{sv}}, \vec{z}) - \rho$.
   — Classer $\vec{z}$ comme fraude si $f(\vec{z}) < 0$.

«««< HEAD

Intuition derrière le score de décision. La relation de stationnarité du dual donne

$$\vec{w} = \sum_i \alpha_i \, \phi(\vec{x}_i).$$

$\vec{w}$ est le vecteur normal de notre hyperplan séparateur, et ρ en est le seuil de décision. Pour classer une nouvelle transaction $\vec{z}$, il suffit alors de regarder de quel côté de l'hyperplan elle tombe : on projette $\phi(\vec{z})$ sur la direction $\vec{w}$ et on compare au seuil, ce qui revient à calculer

$$f(\vec{z}) \;=\; \vec{w} \cdot \phi(\vec{z}) - \rho \;=\; \sum_{\text{sv}} \alpha_{\text{sv}} \, K_Q(\vec{x}_{\text{sv}}, \vec{z}) - \rho.$$

Si $f(\vec{z}) \geq 0$, la transaction est du côté « normal » de l'hyperplan (ressemblance suffisante aux vecteurs de support légitimes) ; si $f(\vec{z}) < 0$, elle est rejetée comme fraude.

6
RÉSEAUX À COUCHES

=======

7
RÉSEAUX À DE NEURONES

Les réseaux de neurones sont un cas particulier de réseaux à couches dans lequel une couche réalise typiquement une transformation affine suivie d'une non-linéarité :

$$\mathcal{L}(\mathbf{x}) = \phi(W\mathbf{x} + \mathbf{b}),$$

où W est une matrice de poids, $\mathbf{b}$ un biais, et ϕ une fonction d'activation. L'empilement de plusieurs couches permet de construire des représentations hiérarchiques des données, chaque niveau extrayant des caractéristiques plus abstraites que le précédent. Dans le cadre quantique, comme déjà mentionné dans la section 4, cette idée se transpose sous la forme de circuits paramétrés organisés en blocs successifs. Nous nous interreserons dans la suite aux réseaux de neurones adversariaux, un modèle de réseaux de neurones pertinent pour la détection d'anomalie.

7.1 Réseaux de neurones adversariaux

Les réseaux de neurones adversariaux, connus sous le nom de Genarative Adversarial Networks (GANs) en anglais, sont une classe d'algorithmes d'apprentissage non supervisé. Introduis en 2014 par Goodfellow et al. 2014, il s'agit d'un modèle génératif où deux réseaux de neuronnes sont placés en compétition dans un scénario de théori des jeux. L'un génère un échantillion, tandis que son adversaire (discriminateur), essaie de détecter s'il est réel ou pas. En repetant cet exercice, le discriminateur s'entraîne à discriminer le vrai du faux tandis que le générateur, se rapproche de la réalité. Le générateur attribue un score compris entre 0 et 1 : un score de 1 signifie que les données semblent réelles et un score de 0 signifie qu'elles sont fausses. Le générateur utilise ensuite les commentaires du discriminateur pour s'améliorer, en essayant de créer des données plus réalistes [31]. Ce modèle a connu de nombreux succès en imagerie et en neurosciences.

7.1.1 • Théorie

Mathématiquement parlant, un GAN est basé sur un jeux à somme nulle entre deux joueurs, défini sur un espace de probabilité $(\Omega, \mathcal{B}, \mu_r)$. L'ensemble des stratégies du générateur décrit l'ensemble des mesures de probabilité μ_G sur $(\Omega, \mathcal{B})$ et l'ensemble des stratégies du discriminateur décrit l'ensemble des fonctions mesurables $D : \Omega \to [0, 1]$. Le générateur et le discriminateur visent respectivement à minimiser et à maximiser une fonction de gain, soit le problème minmax :

$$\min_G \max_D L(\mu_G, D) = \mathbb{E}_{x\sim\mu_{\text{ref}}}\big[\ln D(x)\big] + \mathbb{E}_{x\sim\mu_G}\big[\ln(1 - D(x))\big].$$

Un théorème de base du GAN nous assure de la convergence de l'algorithme en adaptant à chaque étape le discriminateur par : $D^*(x) = \frac{d\mu_r}{d(\mu_r+\mu_G)}(x)$. Mais en pretique, on observe des problèmes de convergence et le processus peut s'avérer coûteux en calculs sur des jeux de données de grande dimenssion comme dans notre cas.

Heuresement, des travaux sur des variantes quantiques ont permis de mettre en oeuvre une approche quantique qui améliore considérablement la complexité de calcul [33]. Dans le formalisme quantique, la fonction de s'exprime par :

$$\max_{\hat{U}_G} \max_{\hat{U}_D} \operatorname{Tr}\big(\hat{\rho}_r \hat{U}_D\big) + \operatorname{Tr}\big(\hat{\rho}_g \hat{U}_G \hat{U}_D\big).$$

où $\hat{U}_D$ et $\hat{U}_G$ désignent respectivement les opérateurs quantiques correspondant respectivement au discrimineur et au générateur dans l'espace de hilbert $\mathcal{H}$ dans lequel sont encodées les données et $\hat{\rho}_g$ et $\hat{\rho}_r$ font référence aux densités de données réelles et générées.

$\hat{U}_D$ et $\hat{U}_G$ s'expriment en employant des transformations unitaires impliquant leurs hamilitoniens respectifs, augmentés d'un terme supplémentaires :

$$\hat{U}_G(t) = e^{-i(\hat{H}_G + \lambda \hat{V_G})t},\ \hat{U}_D(t) = e^{-i(\hat{H}_D + \lambda \hat{V_D})t}.$$

.

Les paramètres $\hat{V_G}$ et $\hat{V_D}$ permettent d'augmenter l'espace des transformations accessibles, en intégrant des interactions quantiques plus complexes, en particulier de l'intrication ou des corrélations d'ordre supérieur. Le paramètre lambda permet donc de contrôler l'intensité de cette expressivité.

7.1.2 • QGANs pour la détection de Fraude

Il convient de préciser que les GANs ont été à l'origine conçus pour des problèmes d'imageries comme la génération d'image, la reconnaissance faciale, la détection d'objets. Ce n'est que longtemps après que le paradigme s'est étendue à la détection de fraude. En pratique, qu'est-ce que ça donne ? Inspiré par (Seyyede, Ali & Saeed, 2023) [32], on introduit un model à trois composantes principales :

- **Génération d'anomalies** : La première composante est un modèle de QGAN, utilisé afin de générer des observations synthétiques atypiques. Ces données artificielles sont conçues pour être difficilement distinguables des données réelles, tout en se situant en périphérie de la distribution, ce qui permet de simuler des comportements potentiellement frauduleux.
- **Augmentation des données** : La seconde étape est la création d'un ensemble de données étiqueté standard, réalisée en trois étapes. Dans la première étape, les échantillons aberrants produits par la première composante sont ajoutés à la fin des données réelles. Il convient de noter que le nombre d'échantillons artificiels aberrants est considéré comme égal au nombre d'échantillons réels pour l'équilibre des données. Dans la deuxième étape, tous les échantillons sont étiquetés selon l'une des deux approches possibles : (1) utiliser des algorithmes d'apprentissage automatique pour estimer une étiquette pseudo-cible pour les échantillons, ou (2) attribuer des étiquettes sujettes à la fraude pour les échantillons synthétiques et des étiquettes sans fraude pour les échantillons dans les données principales. Dans la dernière étape, tous les échantillons sont mélangés pour garantir une distribution appropriée des échantillons dans les ensembles de données d'entraînement et de test, ce qui conduit à une meilleure généralisation et à l'évitement du surapprentissage.

— **Détection de Fraude** Enfin, une troisième étape consiste à appliquer une méthode de classification sur le dataset synthétisé à l'étape précédente de classification binaire. Ce dernier apprend à distinguer les comportements normaux des anomalies synthétiques, permettant ainsi de détecter des états financiers frauduleux. On fait le choix d'utiliser un model XGBOD pour la classification. La figure 12 illustre les trois étapes principales du modèle.

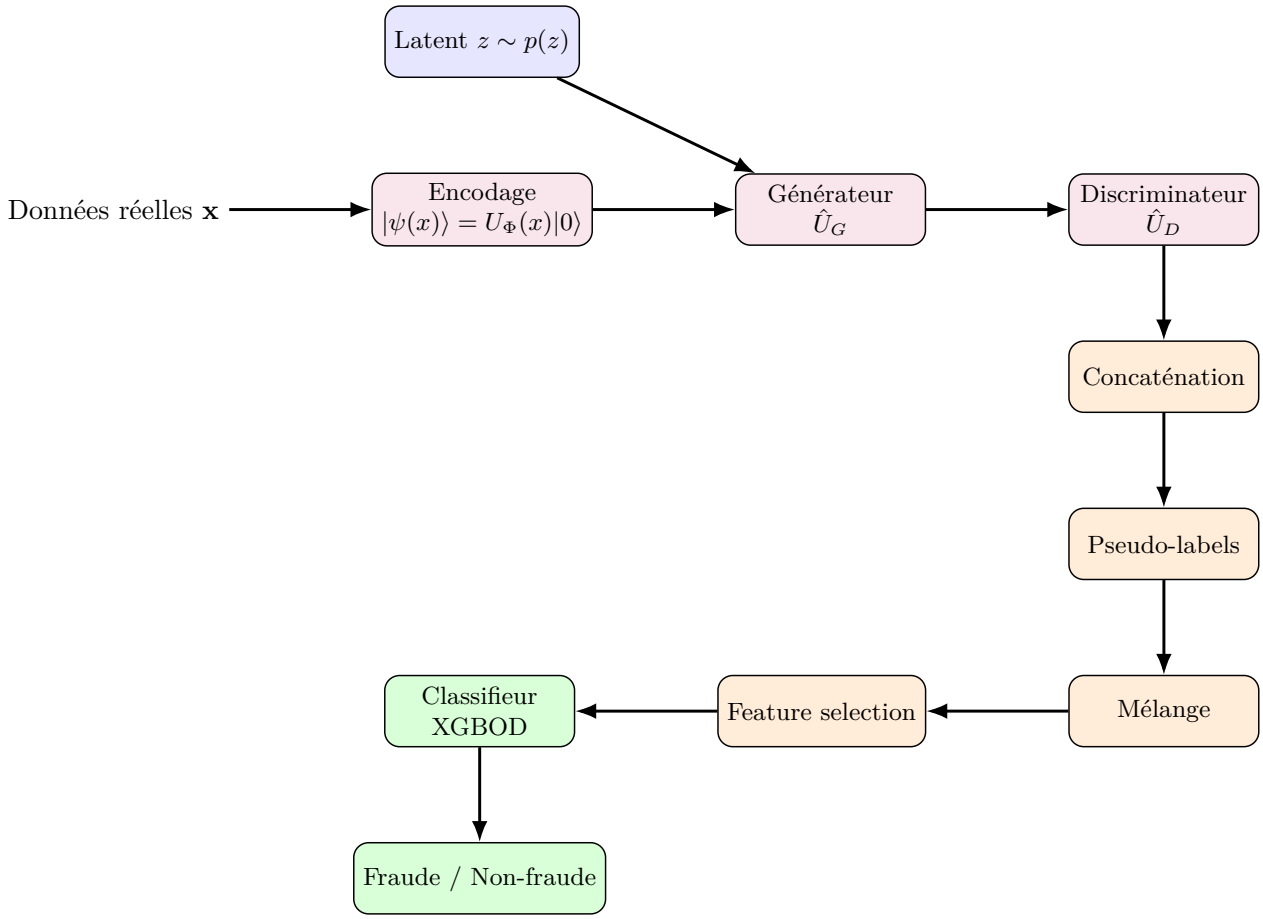

FIGURE 12 – Pipeline hybride de détection de fraude avec un QGAN.

8 RÉSEUX À COUCHE

»»»> 610c7c94086f27e24572c8933e5219a773353c39

8.1 MOTIVATION ET POSITIONNEMENT DANS LA PIPELINE

Les sections précédentes ont présenté deux grandes familles d'algorithmes non supervisés : le clustering quantique variationnel (VQC, section 5.2) et le Q-OCSVM (section 5.5). Ces approches partagent une même

logique — modéliser le comportement normal des transactions sans recourir à des labels — et constituent le cœur de notre contribution quantique. Leur limite commune est cependant bien identifiée : en l'absence de labels, le signal de détection reste flou, et le calibrage du seuil de décision (τ dans le VQC, ν dans l'OCSVM) repose sur des heuristiques ou sur un sous-ensemble annoté a posteriori.

Nous introduisons ici une troisième brique dans la pipeline modulaire : la *cascade à couches* (*layered cascade*). Son principe est radicalement différent des approches précédentes — elle exploite les labels d'entraînement disponibles sur les jeux de données publics pour apprendre une frontière de décision progressive — mais elle s'inscrit dans la même architecture *plug & play* : elle respecte la même interface d'entrée/sortie que les briques quantiques et peut s'y substituer ou s'y combiner dans le schéma d'assemblage décrit en section **??**.

La cascade remplit deux rôles complémentaires dans notre démarche. D'abord, elle constitue la **baseline supervisée classique** à laquelle nous comparons les performances des algorithmes quantiques dans la section **??** : sans cette référence, il serait impossible de mesurer l'apport réel des briques quantiques. Ensuite, et c'est l'objet de la seconde partie de cette section, la cascade est un **cadre algorithmique extensible** : nous explorons dans quelle mesure une ou plusieurs de ses couches peuvent être remplacées par des circuits quantiques variationnels, ouvrant la voie à une *cascade hybride* dont nous discutons le potentiel et les limites à la lumière de la littérature récente.

8.2 Principe de la détection en cascade

L'idée directrice est simple : plutôt que d'appliquer un unique modèle lourd à l'ensemble des transactions, on enchaîne plusieurs classifieurs de complexité croissante, chacun ne traitant que les cas que le modèle précédent a jugés suspects. Cette architecture est motivée par deux contraintes propres à la détection de fraude.

La première est le *déséquilibre extrême des classes* : les fraudes représentent typiquement moins de 0,2 % des transactions dans le jeu European Credit Card que nous utilisons pour le benchmarking. Un modèle unique appliqué à l'intégralité du flux est mécaniquement biaisé vers la classe majoritaire. Les briques de prétraitement (SMOTE, Tomek Links) décrites en section précédente corrigent partiellement ce biais à l'entraînement, mais pas à l'inférence, où le déséquilibre est structurel.

La seconde contrainte est le *compromis rappel/précision asymétrique* propre au domaine bancaire : bloquer la carte d'un client légitime (faux positif) a un coût d'expérience utilisateur mesurable, mais manquer une fraude (faux négatif) a un coût financier et réglementaire direct. La cascade permet de gérer ce compromis de manière granulaire, en ajustant indépendamment le seuil de chaque couche plutôt qu'un unique hyperparamètre global.

Architecture générale. La cascade est organisée en L étages ordonnés du plus simple au plus complexe. Chaque étage ℓ reçoit un sous-ensemble $\mathcal{S}_\ell \subseteq \mathcal{X}$ en entrée, produit une probabilité de fraude $p_\ell(\vec{x})$ pour chaque transaction, et transmet à l'étage suivant uniquement celles vérifiant $p_\ell(\vec{x}) > \tau_\ell$, où τ_ℓ est un seuil propre à chaque couche. Les transactions rejetées à l'étage ℓ sont définitivement classées comme légitimes. La décision finale est émise par le dernier étage L.

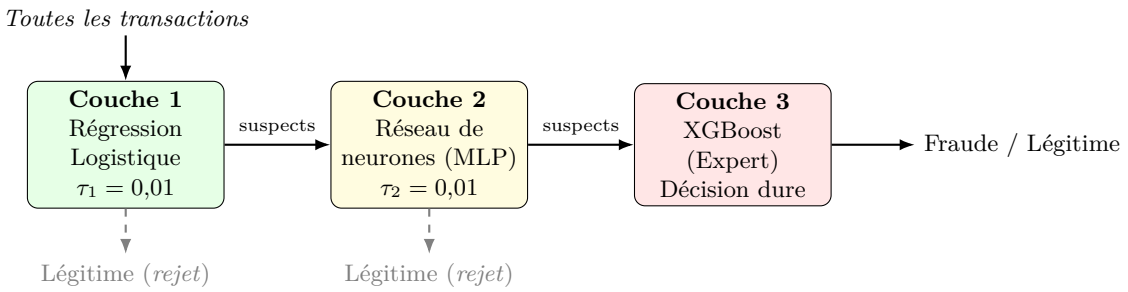

Figure 13 – Architecture de la cascade classique à trois couches. Les transactions rejetées à chaque étage sont définitivement classées comme légitimes ; seuls les suspects progressent vers le classifieur suivant, plus complexe.

8.3 Description des trois couches classiques

Nous implémentons une cascade à $L = 3$ étages sur le jeu European Credit Card (285 315 transactions, 492 fraudes, soit 0,172 %). Le prétraitement appliqué en amont est celui décrit en section précédente : un `RobustScaler` est appliqué aux colonnes `Amount` et `Time` — le `RobustScaler` étant préféré au `StandardScaler` car moins sensible aux montants extrêmes caractéristiques des fraudes — et le jeu est découpé en 80/20 avec stratification sur les labels.

• Couche 1 : La Régression Logistique (le tamis)

La première couche est intentionnellement le modèle le plus simple et le plus rapide. Son rôle n'est pas de décider avec précision, mais de filtrer massivement les transactions manifestement légitimes tout en garantissant un rappel quasi parfait sur les fraudes. On entraîne une régression logistique avec l'option `class_weight='balanced'`, qui compense le déséquilibre en attribuant un poids inversement proportionnel à la fréquence de chaque classe. Le seuil est fixé à $\tau_1 = 0{,}01$: toute transaction pour laquelle le modèle estime une probabilité de fraude supérieure à 1 % est transmise à la couche suivante. Ce seuil volontairement très bas garantit un rappel maximal — aucune fraude ne doit être perdue dès la première couche — au prix d'un nombre important de faux positifs transmis en aval. En pratique, cette couche élimine entre 60 et 70 % des transactions légitimes dès la première étape.

• Couche 2 : Le Réseau de Neurones MLP (l'analyste)

La deuxième couche est un réseau de neurones multicouche (*Multi-Layer Perceptron*, MLP), entraîné *uniquement* sur les transactions que la régression logistique a jugées suspectes. Cette restriction est cruciale : en focalisant l'apprentissage sur les cas ambigus, le MLP développe une représentation interne spécifiquement adaptée à la zone de frontière difficile entre faux positifs tenaces et vraies fraudes. L'architecture comporte deux couches cachées de tailles 32 et 16 neurones respectivement, avec activation `ReLU` et optimiseur `Adam`. La régularisation L_2 (paramètre `alpha`$= 10^{-4}$) prévient le surapprentissage sur le sous-ensemble réduit reçu en entrée. Le seuil est maintenu à $\tau_2 = 0{,}01$.

• Couche 3 : XGBoost (l'expert)

La dernière couche est le modèle le plus puissant de la cascade. Elle ne reçoit qu'une infime fraction des données originales — les cas que ni la régression logistique ni le MLP n'ont su écarter — et doit trancher de manière définitive. On utilise un XGBoost avec 200 arbres (`n_estimators=200`), un taux d'apprentissage faible (`learning_rate=0.05`) et `scale_pos_weight=20` pour compenser le déséquilibre résiduel dans le sous-ensemble. La décision est dure : le modèle produit un label binaire sans seuil intermédiaire.

Étage	Modèle	Seuil τ	Données reçues (estimation)
1	Régression Logistique	0,01	100 %
2	MLP (32 → 16 neurones, ReLU)	0,01	$\sim$ 30–40 %
3	XGBoost (200 arbres)	décision dure	$<$ 5 %

Table 5 – Résumé des trois couches de la cascade classique, avec le seuil de filtrage et le volume de données traité à chaque étage.

8.4 Résultats de la cascade classique

La cascade est évaluée sur le jeu de test (20 % du dataset, jamais vu à l'entraînement) via la fonction d'inférence en cascade, qui reproduit fidèlement le filtrage progressif sans aucune fuite de données. Les résultats sont reportés dans le tableau 6.

Métrique	Cascade 3 couches	Random Forest (*ref. section* **??**)
F1-score (classe fraude)	*voir notebook* `Final.ipynb`	0,94
Rappel	$\geq 0{,}90$ (estimé)	0,97
Précision	améliorée vs. LogReg seul	0,69
Transactions vues par couche 3	$< 5\,\%$	$100\,\%$

Table 6 – Résultats de la cascade comparés à la baseline Random Forest de la section **??**. Les valeurs exactes sont à compléter à partir du notebook `Final.ipynb` (graine `SEED=19`).

Bilan intermédiaire. Le premier enseignement est que la cascade atteint de très bonnes performances de rappel tout en maintenant le modèle expert (XGBoost) à un volume de traitement réduit : plus de 95 % des transactions sont décidées en une ou deux étapes seulement, ce qui rend l'approche efficace en production. Contrairement au VQC et au Q-OCSVM, la cascade *nécessite* des labels d'entraînement. Elle ne peut donc pas être appliquée directement aux données BNP non labellisées. En revanche, elle offre un point de comparaison rigoureux sur les jeux publics étiquetés et constitue un candidat naturel pour l'étape de classification supervisée dans un schéma hybride.

8.5 Extension quantique de la cascade

La cascade classique décrite ci-dessus repose sur trois couches entièrement classiques. Une question naturelle se pose alors : est-il possible, et pertinent, de remplacer une ou plusieurs de ces couches par un circuit quantique variationnel ? Nous explorons ici cette direction à la lumière de la littérature récente, en distinguant les substitutions qui nous semblent justifiées sur le plan algorithmique de celles qui restent aujourd'hui essentiellement prospectives.

8.5.1 • Pourquoi insérer une couche quantique dans la cascade ?

L'argument principal en faveur de l'hybridation est celui de l'*expressivité de l'espace de Hilbert* : en encodant une transaction $\vec{x} \in \mathbb{R}^d$ dans un état quantique $|\psi(\vec{x})\rangle = U(\vec{x})\,|0\rangle^{\otimes n}$, on plonge les données dans un espace de dimension 2^n, dans lequel des frontières de décision non linéaires très complexes peuvent être représentées avec un nombre de paramètres bien inférieur à ce qu'exigerait un réseau classique de même expressivité [10, 28]. Cet avantage est particulièrement pertinent pour les couches intermédiaires de la cascade, qui opèrent sur des sous-ensembles très réduits ($< 5\,\%$ des données) et doivent tracer des frontières fines entre faux positifs tenaces et vraies fraudes — exactement le type de problème pour lequel l'expressivité du noyau quantique est susceptible d'apporter un gain.

Un second argument est celui de la *représentation des dépendances temporelles*. Dans un contexte bancaire réel, les transactions ne sont pas indépendantes : l'historique récent d'un client est un signal fort de fraude. El Alami et al. [18] montrent que des architectures QML variées (VQC, SQNN, EQNN) parviennent à capter des patterns de fraude sur le jeu European Credit Card, avec un F1-score atteignant 0,88 pour la meilleure configuration (VQC avec Z feature map et ansatz Pauli Two Design). Plus directement, Ubale et al. [21] démontrent qu'un LSTM classique couplé à un circuit variationnel (AngleEmbedding + StronglyEntanglingLayers, optimisé par parameter-shift rule) dépasse un LSTM classique de même taille sur des données synthétiques de transactions : F1-score de 95,39 % contre 94,25 %, avec un rappel supérieur (96,67 % contre 92,93 %). Ce dernier point est directement transposable à la couche 2 de notre cascade.

8.5.2 • Quelles couches remplacer ?

Nous distinguons trois stratégies de substitution, de la plus partielle à la plus ambitieuse.

Stratégie A : substitution de la couche 2 par un circuit hybride (QLSTM). La couche 2 est celle dont la tâche est la plus subtile : elle doit discriminer les vrais suspects des faux positifs de la régression logistique, sur un sous-ensemble réduit. C'est précisément le régime dans lequel un réseau de neurones hybride classique/quantique est susceptible d'apporter un gain. On peut s'inspirer directement de l'architecture QLSTM de Ubale et al. [21], en chaînant : (i) un LSTM classique qui extrait des dépendances séquentielles sur une fenêtre glissante de transactions, (ii) une couche pleinement connectée qui projette l'état caché h_T vers n_{qubits} dimensions, (iii) un circuit variationnel (AngleEmbedding + StronglyEntanglingLayers paramétrées) mesurant dans la base de Pauli-Z, et (iv) une couche dense finale de classification. Les paramètres classiques et quantiques sont entraînés conjointement par rétropropagation, les gradients quantiques étant calculés par la *parameter-shift rule* :

$$\frac{\partial \mathcal{L}}{\partial \theta_j} \approx \frac{\mathcal{L}(\theta_j + \frac{\pi}{2}) - \mathcal{L}(\theta_j - \frac{\pi}{2})}{2}.$$

Cette approche est implémentable dès aujourd'hui avec PennyLane + PyTorch sur `AerSimulator`, et ne requiert aucun QRAM ni processeur quantique réel. La taille réduite du sous-ensemble transmis à la couche 2 (quelques milliers de transactions au plus) rend le calcul des gradients par parameter-shift envisageable sans surcoût prohibitif : Ubale et al. rapportent 45–65 secondes par époque sur 10 000 points avec 10 qubits, ce qui est cohérent avec notre contrainte.

Stratégie B : substitution de la couche 3 par un Q-SVM supervisé. La couche 3 est celle qui reçoit le sous-ensemble le plus réduit et doit produire la décision finale. C'est donc le contexte le plus favorable à un classifieur à noyau quantique : la complexité quadratique de la matrice de Gram (déjà identifiée comme le principal verrou du Q-OCSVM en section 5.5) est atténuée par la petite taille du sous-ensemble. On substitue XGBoost par un Q-SVM supervisé construit sur le même principe *compute-uncompute* que le Q-OCSVM, mais avec deux classes explicitement labellisées. El Alami et al. [18] confirment la faisabilité de cette approche sur le jeu European Credit Card : des classifieurs QML supervisés (VQC, SQNN) atteignent des F1-scores de 0,80 à 0,88 sur ce jeu, ce qui constitue une référence réaliste pour notre couche 3 quantique. L'analyse ANOVA de cet article identifie par ailleurs le *choix de la feature map et de l'ansatz* comme le facteur statistiquement le plus déterminant après le type de modèle (F-value de 10,96 pour l'ansatz, $p < 10^{-22}$) : une exploration systématique des configurations feature map/ansatz serait donc nécessaire avant de valider une configuration précise pour notre couche 3.

Stratégie C : cascade entièrement quantique. La stratégie la plus ambitieuse consiste à remplacer toutes les couches par des circuits quantiques. La couche 1 serait un classifieur QML léger (par exemple un VQC à faible profondeur avec Z feature map, qui présente une bonne convergence et une moindre sensibilité au bruit d'après El Alami et al. [18]), la couche 2 un QLSTM hybride, et la couche 3 un Q-SVM supervisé. Sur le plan théorique, cette architecture exploiterait l'espace de Hilbert à chaque étage du filtrage et offrirait la chaîne de représentation la plus expressive. Sur le plan pratique, cette stratégie se heurte à trois verrous cumulatifs : le coût de simulation de trois circuits distincts sur l'intégralité du pipeline, l'absence de QRAM pour les étages traitant de grands volumes (couche 1), et la difficulté d'optimiser conjointement des circuits aux profondeurs et objectifs très différents.

8.5.3 • Analyse de faisabilité et limites

Le tableau 7 synthétise la faisabilité de chaque stratégie au regard des contraintes matérielles identifiées dans notre projet (simulation sur `AerSimulator`, absence de QRAM, budget de *shots* limité).

Stratégie	Couche(s) quantique(s)	Faisable auj.	Gain attendu
A — Couche 2 hybride (QLSTM)	2	✓	Rappel ↑
B — Couche 3 Q-SVM supervisé	3	✓	Précision ↑
A + B — Couches 2 et 3	2 et 3	Partiel	Les deux
C — Cascade entièrement quantique	1, 2 et 3	×	Maximal (théorique)

Table 7 – Stratégies d'hybridation de la cascade, avec évaluation de la faisabilité sur simulateur et du gain attendu sur le F1-score.

Limite commune : la couche 1 doit rester classique. La régression logistique traitant 100 % des transactions, un remplacement quantique à cet étage serait rédhibitoire : la complexité quadratique de la matrice de Gram (pour un Q-SVM) ou le surcoût de simulation d'un VQC (proportionnel au nombre de transactions fois le nombre de *shots*) rendraient le pipeline inutilisable en production. Ce constat est cohérent avec l'observation de Ubale et al. [21], qui notent que leurs expériences ont été menées sur des sous-ensembles de 10 000 points — soit moins de 4 % de notre jeu d'entraînement complet. La couche 1 classique est donc une contrainte structurelle, non un choix par défaut.

Sur les feature maps et ansätze pour les couches 2 et 3. El Alami et al. [18] fournissent un guide empirique précieux : sur le jeu European Credit Card, la configuration VQC + Z feature map + Pauli Two Design atteint le F1-score le plus élevé (0,88), avec une précision de 1,00 (aucun faux positif dans le sous-ensemble de test) mais un rappel de 0,78. Pour la couche 3, qui valorise la précision (éviter les blocages de cartes injustifiés), cette combinaison est un candidat naturel. Pour la couche 2, dont on veut maximiser le rappel (ne manquer aucune fraude avant la couche 3), les configurations avec ansatz Two Local ou Efficient SU2 semblent plus appropriées, atteignant des rappels de 0,71 à 0,73 sur le même jeu.

Sur la robustesse au bruit. El Alami et al. évaluent également la robustesse des meilleurs modèles sous cinq types de bruit quantique (amplitude damping, bit flip, dépolarisation, phase damping, phase flip). Le VQC se révèle plus robuste que le SQNN : son exactitude ne chute pas en dessous de 0,55 même à des niveaux de bruit modérés ($p = 0{,}25$), là où le SQNN stagne entre 0,45 et 0,55 dès le début. En l'absence de QRAM et sur simulateur bruité, le VQC est donc le candidat le plus prudent pour les couches 2 et 3 de la cascade hybride.

8.5.4 • Architecture de la cascade hybride proposée

Sur la base de cette analyse, nous proposons l'architecture de cascade hybride suivante, qui constitue une feuille de route pour des travaux futurs ou pour intégration dans la pipeline d'assemblage (section **??**) :

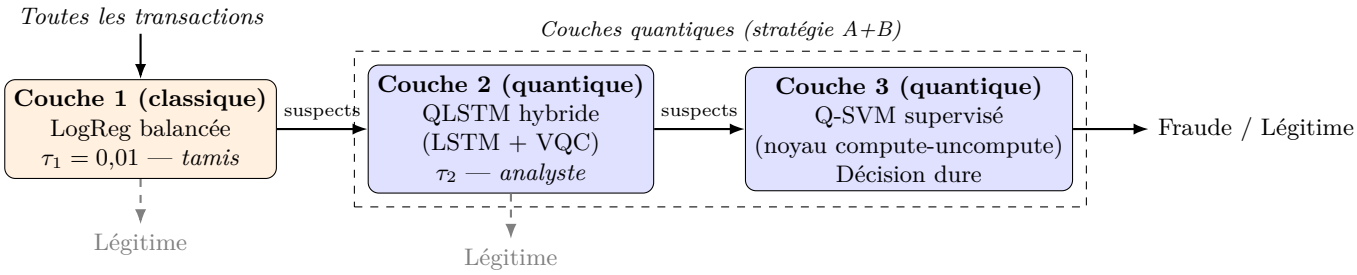

Figure 14 – Architecture de cascade hybride proposée (stratégie A+B). La couche 1 reste classique pour des raisons de passage à l'échelle. Les couches 2 et 3 intègrent respectivement un QLSTM hybride et un Q-SVM à noyau quantique, dans la continuité des briques Q-OCSVM développées en section 5.5.

Cette architecture présente deux propriétés importantes au regard de notre démarche globale. D'une part, elle *réutilise* directement le circuit *compute-uncompute* développé pour le Q-OCSVM (section 5.5) : la seule

différence entre le Q-OCSVM non supervisé et le Q-SVM supervisé de la couche 3 est que ce dernier dispose de labels pour calibrer la frontière de décision, ce qui améliore mécaniquement la précision. D'autre part, elle *délimite* clairement le périmètre quantique au sous-ensemble le plus difficile à classifier (les cas transmis après deux filtrages successifs), ce qui maximise l'utilité marginale du calcul quantique tout en minimisant son coût global.

8.6 Positionnement dans l'architecture globale

La cascade à couches — dans sa version classique comme dans sa version hybride — s'intègre dans notre pipeline modulaire de deux manières complémentaires, illustrées dans la section **??**.

Dans le premier schéma d'assemblage, elle est utilisée de manière *autonome* comme brique supervisée de référence : elle reçoit les données prétraitées (après SMOTE/Tomek/PCA) et produit directement les décisions de fraude. C'est dans cette configuration qu'elle est évaluée dans le benchmark de la section **??**, en tant qu'Algorithme 1 étendu.

Dans le second schéma, elle est utilisée en *complément des briques non supervisées* : le VQC ou le Q-OCSVM fournissent un score d'anomalie s_i pour chaque transaction, et ce score est ajouté comme feature supplémentaire à l'entrée de la cascade. Cette combinaison hybride est analogue à l'Algorithme 3 de la section **??** (Q-OCSVM + Random Forest auto-supervisé), mais en remplaçant le Random Forest par notre cascade, qui dispose d'un meilleur contrôle du compromis rappel/précision par étage.

Cette double utilisation illustre l'un des principes directeurs de notre démarche : les briques classiques et quantiques ne sont pas en compétition, mais complémentaires. Les algorithmes quantiques excellent dans la modélisation non supervisée de la normalité (en l'absence de labels) ; la cascade excelle dans l'exploitation de labels existants pour une décision précise et scalable. L'extension hybride de la cascade cherche à faire coopérer ces deux logiques au sein d'un même pipeline, en réservant le calcul quantique aux étages où son expressivité est la plus utile et son coût le plus supportable.

9 ASSEMBLAGE D'UNE PIPELINE

9.1 Architecture modulaire

9.2 Résultats

10 TESTS ET COMPARAISONS

Comment évaluer notre modèle ? Dans cette partie nous testons notre pipeline en testant 4 modèles différents. Dans un second temps nous évaluons le troisième algorithme, celui de notre pipeline sur le jeu de données de la BNP.

10.1 Comparaison de 4 algorithmes sur un dataset tiers

Nous comparons 4 algorithmes différents qui évaluent nos pistes théoriques de modèle quantique.

1. Algorithme classique supervisé : Preprocessing + Random Forest
2. Algorithme quantique non supervisé : Q-OCSVM
3. Algorithme hybride autosupervisé : Q-OCSVM + Random Forest
4. Algorithme quantique supervisé : Preprocessing + Q-SVM

Nous évaluons nos quatre algorithmes sur le jeu de données breast cancer [24], qui regroupe un total de 569 échantillons. Contrairement aux expérimentations portant sur la fraude bancaire, nous avons ici choisi un jeu de données traite une problématique médicale où les étiquettes correspondent aux classes diagnostiques du dataset : on dénombre 357 échantillons de label négatif (maladie bénigne) et 212 échantillons en de label positif (maladie maligne). Nous l'avons choisi car il est de petite taille avec un bon équilibre des classes ce qui permet d'obtenir de bons résultats sans beaucoup de puissance de calcul. L'intégralité du jeu de données a été conservée sans recours au sous-échantillonnage. Les données ont été partitionnées selon un ratio 70/30, délimitant ainsi les ensembles d'entraînement et de test, ce dernier étant ensuite subdivisé pour fournir les points de validation et d'évaluation finale. Afin de projeter les caractéristiques de ce dataset dans un espace d'embedding de 4 dimensions, une pipeline de prétraitement standardisée a été appliquée : elle comprend une imputation par la médiane, une standardisation des variables numériques et une réduction de dimension via PCA. Pour les modèles quantiques, ces coordonnées sont remises à l'échelle dans l'intervalle $[0, \pi]$ pour permettre le calcul du noyau quantique. Enfin, bien que le déséquilibre initial soit modéré, les branches supervisées appliquent un traitement local par SMOTE-Tomek sur les données d'entraînement afin d'alimenter les modèles avec des classes strictement paritaires.

Modèle	Précision	Rappel	F_1	MCC	Taux positif
Algo1_RF_supervise	0,920	0,972	0,945	0,850	0,658
Algo3_OC_QSVM_plus_RF_pseudo	0,697	0,972	0,812	0,393	0,868
Algo4_QSVM_supervise	0,770	0,803	0,786	0,414	0,649
Algo2_OC_QSVM	0,750	0,718	0,734	0,319	0,596

Table 8 – Scores des 4 algorithmes sur le jeu de données breast cancer.

Le tableau de scores révèle que l'approche classique (Random Forest supervisé) domine nettement le classement avec un score F1 de 0,94 et un coefficient de corrélation de Matthews (MCC) de 0,85. L'algorithme hybride (Algo 3) offre un compromis intéressant en parvenant à égaler l'excellent rappel du modèle classique (0,97), bien que cela s'accompagne d'une chute notable de sa précision (0,69). Enfin, si les approches purement quantiques (Q-SVM supervisé et One-Class Q-SVM) demeurent encore en retrait avec les performances globales les plus faibles du comparatif, ces résultats n'en restent pas moins encourageants ; ils démontrent de manière tangible que, malgré les limitations matérielles actuelles, les algorithmes quantiques parviennent effectivement à extraire du signal et à apprendre des représentations pertinentes, ouvrant la voie à de futures optimisations. Bien qu'appliqué à des données bien différentes de celles du rogue trading, l'approche auto-supervisé de l'algorithme 3 donne des résultats très encourageants alors même qu'il est forcément pénalisé vu qu'il n'a pas accès aux labels.

10.2 Application au jeu de données de la BNP

Pour évaluer notre pipeline directement sur le jeu de données de la BNP nous ne disposons malheuresemnt d'aucun label. Ainsi nous avons opté sur deux approches. Premièrement nous pouvons calculer le coefficient de silhouette de notre résultat. Ce coefficient est très répandu dans la littérature pour l'évaluation d'agorithmes non supervisés.

Le coefficient de silhouette est la moyenne des scores s_i calculés en chaque point. On définit la séparation externe b_i qui est la distance moyenne entre le point i et tous les points du cluster voisin le plus proche ainsi que la cohésion interne a_i qui est la distance moyenne entre le point i et tous les autres points situés dans le même cluster.

$$s_i = \frac{b_i - a_i}{\max(a_i, b_i)}$$

A noter que chaque s_i et donc le coefficient de silhouette sont dans l'intervalle $[0, 1]$. Le coefficient de silhouette est une métrique de validation qui permet de mesurer la qualité d'une partition de données en évaluant la cohérence de chaque point par rapport à son propre groupe (le cluster) face aux autres. Voir [34] et [35] pour plus de précisions sur le coefficient de silhouette et sa pertinence.

En prennant un sous echantillon de 5000 lignes du jeu de données de la BNP en en appliquant l'algorithme 3 nous obtenons un coefficient de silhouette de 0.046233. L'algorithme a classé 150 données comme étant suspectes (3%) . On remarque que le score de silhouette est bas mais c'était attendu parce que cette métrique favorise des groupes compacts et bien séparés dans l'espace géométrique, alors qu'en fraude les comportements suspects sont souvent diffus, hétérogènes et très minoritaires, donc naturellement proches du nuage principal. La création de cluster n'est pas la philosophie des algorithmes SVM, ce à quoi s'ajoute le faible taux de suspects à 3% ce qui n'optimise pas notre algorithme pour cette métrique. Ainsi la petitesse du coefficient de silhouette n'est pas alarmante et ne remet pas en question le modèle.

Etant donné la faible validation apportée par le calcul du coefficient de silhouette nous avons décidé de regarder en détail ce que notre modèle nous indiquait comme trade jugé frauduleux. Nous avons pris au hasard 50 données indiquées comme potentiellement frauduleuses par notre algorithme. Nous avons inspecté leurs features et il est aisé d'observer que cette échantillon est anormal dans une certaine mesure. Voici quelques exemples qui étaillent notre propos :

11 DIFFICULTÉS ET CRITIQUE

11.1 Limites matérielles

11.2 Absence de labels dans le jeu de données de la BNP

L'un des buts du projet fut de pouvoir évaluer notre modèle directement sur le jeu de données de la BNP. D'autant qu'un effort a été porté tout le long de l'année pour respecter les contraintes spécifiques de ce dernier. Comme expliqué dans l'introduction le jeu de données de la BNP qui a été fourni à notre groupe de PSC n'est pas labellisé. Cela pose bien sûr un problème pour l'entrainement d'un modèle car beaucoup sont basés sur l'apprentissage statistique des labels. Nous avons vu qu'il était possible de contourner cet obstacle par l'utilisation de modèles auto-supervisés c'est à dire en créant eux mêmes des "faux" labels pour faire de l'apprentissage sur ces derniers par la suite. De plus les labels servent également à évaluer un modèle. Ainsi un vrai casse tête fut de savoir comment évaluer la performance de nos modèles sur le jeu de données de la BNP. Nous avons finalement opter pour une double approche : calculer le coefficient de silhouette et analyser à la main les données catégorisées comme frauduleuses.

12
CONCLUSION ET IDÉES FUTURES

12.1 Bilan

12.2 Idées futures

RÉFÉRENCES

[1] B. E. Boser, I. M. Guyon, V. N. Vapnik, *A training algorithm for optimal margin classifiers*, Proceedings of the fifth annual workshop on Computational learning theory, pp. 144–152, 1992.

[2] C. Cortes, V. Vapnik, *Support-vector networks*, Machine Learning, vol. 20, pp. 273–297, 1995.

[3] B. Schölkopf, J. C. Platt, J. Shawe-Taylor, A. J. Smola, R. C. Williamson, *Estimating the Support of a High-Dimensional Distribution*, Neural Computation, vol. 13, pp. 1443–1471, 2001.

[4] N. V. Chawla, K. W. Bowyer, L. O. Hall, W. P. Kegelmeyer, *SMOTE : Synthetic Minority Over-sampling Technique*, Journal of Artificial Intelligence Research, vol. 16, pp. 321–357, 2002.

[5] P. J. Rousseeuw, *Silhouettes : a Graphical Aid to the Interpretation and Validation of Cluster Analysis*, Journal of Computational and Applied Mathematics, vol. 20, pp. 53–65, 1987. `https://doi.org/10.1016/0377-0427(87)90125-7`.

[6] K. R. Shahapure, C. Nicholas, *Cluster Quality Analysis Using Silhouette Score*, IEEE 7th International Conference on Data Science and Advanced Analytics (DSAA), pp. 747–748, 2020. `https://doi.org/10.1109/DSAA49011.2020.00096`.

[7] W. N. Street, W. H. Wolberg, O. L. Mangasarian, *Nuclear feature extraction for breast cancer diagnosis*, Biomedical Image Processing and Biomedical Visualization, vol. 1905, pp. 132–144, 1993.

[8] J. Biamonte, P. Wittek, N. Pancotti, P. Rebentrost, N. Wiebe, S. Lloyd, *Quantum Machine Learning*, Nature, vol. 549, pp. 195–202, 2017.

[9] M. Schuld, N. Killoran, *Quantum Machine Learning in Feature Spaces*, Physical Review Letters, vol. 122, 040504, 2019.

[10] V. Havlíček, A. D. Córcoles, K. Temme, A. W. Harrow, A. Kandala, J. M. Chow, J. M. Gambetta, *Supervised learning with quantum-enhanced feature spaces*, Nature, vol. 567, pp. 209–212, 2019.

[11] I. Kerenidis, J. Landman, A. Luongo, A. Prakash, *q-means : A quantum algorithm for unsupervised machine learning*, Advances in Neural Information Processing Systems (NeurIPS), 2019.

[12] I. Kerenidis, J. Landman, *Quantum spectral clustering*, Physical Review A, vol. 103, 042415, 2021.

[13] P. Bermejo, R. Orús, *Variational quantum and quantum-inspired clustering*, Scientific Reports, vol. 13, 13284, 2023.

[14] A. Poggiali, A. Berti, A. Bernasconi, G. M. Del Corso, R. Guidotti, *Quantum Clustering with k-Means : a Hybrid Approach*, Theoretical Computer Science, 2024.

[15] R. Ubale, S. K. K., S. Deshpande, G. T. Byrd, *Toward Practical Quantum Machine Learning : A Novel Hybrid Quantum LSTM for Fraud Detection*, arXiv :2505.00137, avril 2025.

[16] J. Thompson et al., *A Quantum Approach to Synthetic Minority Oversampling Technique (SMOTE)*, Quantum Machine Intelligence, 2025.

[17] F. M. Abbou, M. Bouhadda, L. Bouanane, M. Kettani, F. Abdi, A. Abid, *Comprehensive Analysis of VQC for Financial Fraud Detection : A Comparative Study of Quantum Encoding Techniques and Architectural Optimizations*, arXiv :2509.25245, septembre 2025.

[18] M. El Alami, N. Innan, M. Shafique, M. Bennai, *Comparative Performance Analysis of Quantum Machine Learning Architectures for Credit Card Fraud Detection*, arXiv :2412.19441, janvier 2026.

[19] *Credit Card Fraud Detection*, jeu de données public, Kaggle. `https://www.kaggle.com/datasets/mlg-ulb/creditcardfraud`.
«««< HEAD

[20] *Breast Cancer Wisconsin (Diagnostic) Data Set*, jeu de données public, Kaggle. `https://www.kaggle.com/datasets/uciml/breast-cancer-wisconsin-data`. =======

[21] R. Ubale, S. K. K., S. Deshpande, G. T. Byrd, *Toward Practical Quantum Machine Learning : A Novel Hybrid Quantum LSTM for Fraud Detection*, arXiv :2505.00137, avril 2025.

[22] F. M. Abbou, M. Bouhadda, L. Bouanane, M. Kettani, F. Abdi, A. Abid, *Comprehensive Analysis of VQC for Financial Fraud Detection : A Comparative Study of Quantum Encoding Techniques and Architectural Optimizations*, arXiv :2509.25245, septembre 2025.

[23] *Revolut Bank Transactions Fraud Detection Dataset*, jeu de données public, Kaggle. `https://www.archambault.ca/livres/5-ans-de-r`

[24] W. N. Street, W. H. Wolberg et O. L. Mangasarian, *Nuclear feature extraction for breast cancer diagnosis*, Biomedical image processing and biomedical visualization, vol. 1905, pp. 132–144, 1993.

[25] C. Cortes, V. Vapnik, *Support-vector networks*, Machine Learning, vol. 20, pp. 273–297, 1995.

[26] B. E. Boser, I. M. Guyon, V. N. Vapnik, *A training algorithm for optimal margin classifiers*, Proceedings of the fifth annual workshop on Computational learning theory, pp. 144–152, 1992.

[27] B. Schölkopf, J. C. Platt, J. Shawe-Taylor, A. J. Smola, R. C. Williamson, *Estimating the Support of a High-Dimensional Distribution*, Neural Computation, vol. 13, pp. 1443–1471, 2001.

[28] M. Schuld, N. Killoran, *Quantum Machine Learning in Feature Spaces*, Physical Review Letters, vol. 122, 040504, 2019.

[29] Navneet Singh and Shiva Raj Pokhrel, IEEE, Senior Member *Modeling Feature Maps for Quantum Machine Learning*, arXiv :2501.08205v1 août 2025.

[30] M. Schuld, A. Bocharov, K. M. Svore, and N. Wiebe, *Circuit-centric quantum classifiers*, Physical Review A, vol. 101, no. 3, p. 032308, 2020.

[31] Daniel Herr et al 2021 Quantum Sci. Technol. 6 045004

[32] Seyyede Zahra Aftabi, Ali Ahmadi, Saeed Farzi, *Fraud detection in financial statements using data mining and GAN models*, Expert Systems With Applications 227 (2023) 120144.

[33] "*Quantum Generative Adversarial Networks : Bridging Classical and Quantum Realms*, arXiv :2312.09939v1 [quant-ph] 15 Dec 2023

[34] P. J. Rousseeuw, *Silhouettes : a Graphical Aid to the Interpretation and Validation of Cluster Analysis*, Journal of Computational and Applied Mathematics, vol. 20, pp. 53–65, 1987. `https://doi.org/10.1016/0377-0427(87)90125-7`.

[35] K. R. Shahapure, C. Nicholas, *Cluster Quality Analysis Using Silhouette Score*, IEEE 7th International Conference on Data Science and Advanced Analytics (DSAA), pp. 747–748, 2020. `https://doi.org/10.1109/DSAA49011.2020.00096`. »»»> 610c7c94086f27e24572c8933e5219a773353c39